

符集，其最高位是1。

表26.1列出某些控制字符及其意义。

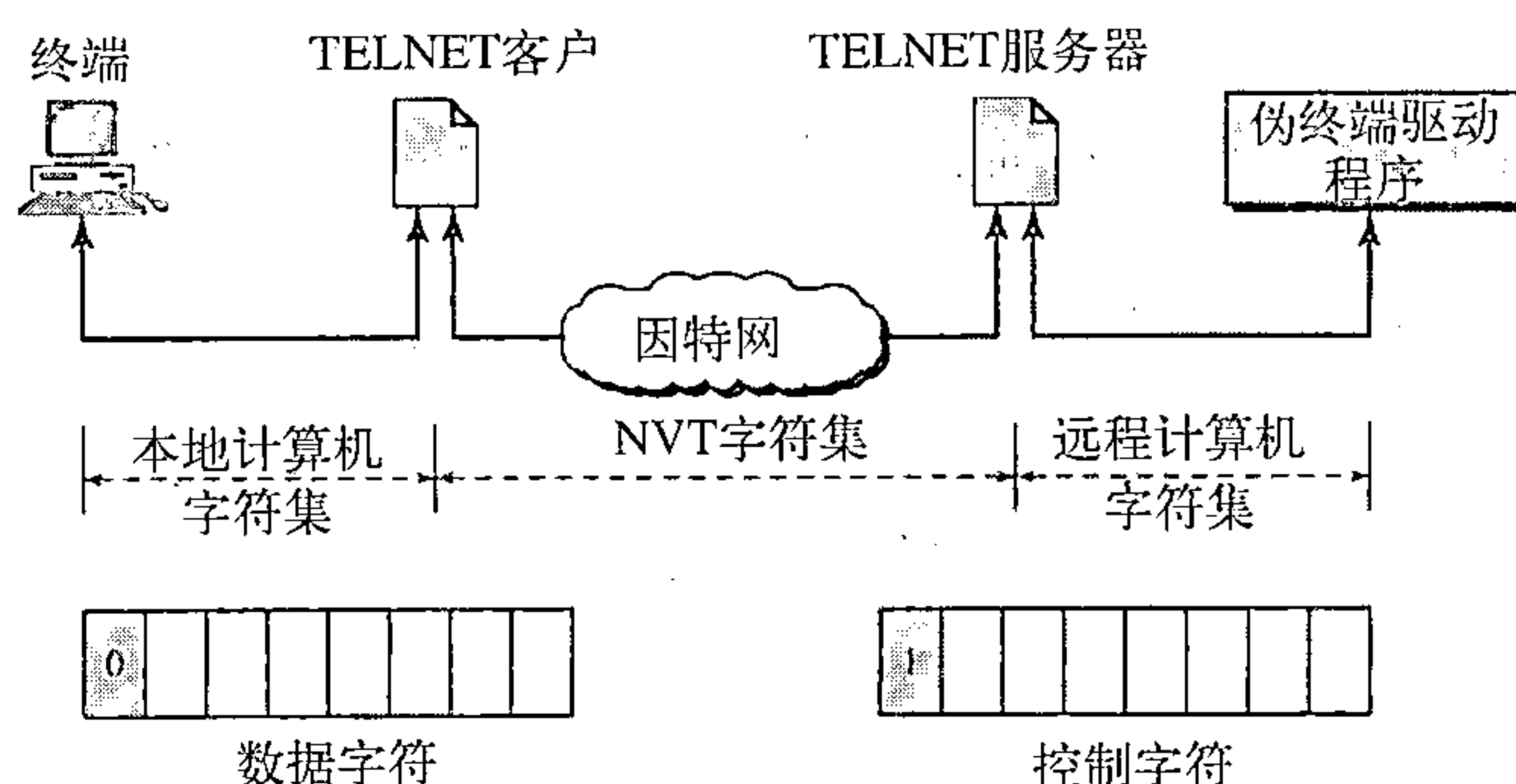


图26.2 NVT的概念

表26.1 一些NVT控制字符

字 符	十进制	二进制	意 义
EOF	236	11101100	文件结束
EOR	239	11101111	记录结束
SE	240	11110000	子选项结束
NOP	241	11110001	无操作
DM	242	11110010	数据标记
BRK	243	11110011	断开
IP	244	11110100	中断过程
AO	245	11110101	异常中止输出
AYT	246	11110110	对方是否还在运行?
EC	247	11110111	擦除字符
EL	248	11111000	擦除行
GA	249	11111001	前进
SB	250	11111010	子选项开始
WILL	251	11111011	同意激活选项
WONT	252	11111100	拒绝激活选项
DO	253	11111101	认可选项请求
DON'T	254	11111110	拒绝选项请求
IAC	255	11111111	作为控制解释 (下一个字节)

嵌入

TELNET仅使用一个TCP连接，服务器使用熟知端口23，而客户使用一个临时端口。使用同一个连接发送数据和控制字符。TELNET做到这点是将控制字符嵌入到数据流中。但是，要将数据与控制字符区别开来，在每个控制字符序列的前面要加上一个特殊的控制字符，它称为作为控制解释（IAC）。例如，假定一个用户想要一个服务器在一个远程服务器上显示一个文件（file1），他键入：

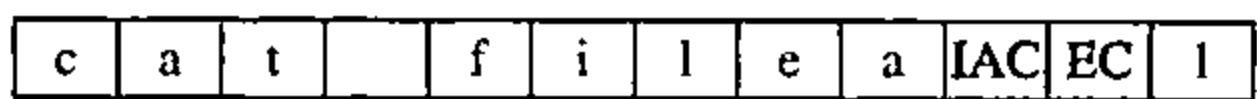
```
cat file1
```

然而，假定文件名输入错（如键入filea而不是file1），那么用户可用退格键进行改正：

```
cat filea<backspace>1
```

但是，在TELNET的默认实现中，用户不能在本地进行编辑，编辑工作必须在远程服务

器上进行。退格字符被转换成两个远程字符 (IAC EC)，它嵌入在数据中发送到远程服务器。图26.3表示了发送给服务器的内容。



在远程终端键入

图26.3 嵌入的例

选项

TELNET让客户与服务器在使用服务之前或期间可以协商选项。对具有更加复杂的终端用户，这些选项还提供额外的特性。使用较简单终端的用户只能使用默认的特性。前面讨论的某些控制字符可用来定义选项，表26.2给出一些常用的选项。

表26.2 选项

代 码	选 项	意 义
0	二进制	解释为8位二进制传输
1	回显	将在一端接收到的数据回显到另一端
3	抑制前进	抑制数据后面的前进信号
5	状态	请求TELNET的状态
6	计时标记	定义计时标记
24	终端类型	设置终端类型
32	终端速率	设置终端速率
34	行方式	改变到行方式

选项协商 为了使用前面提到的任何选项，首先需要在客户与服务器之间进行选项协商 (option negotiation)。为此要使用4个控制字符， 这些字符如表26.3所示。

表26.3 NVT选项协商字符集

字 符	十 进 制	二 进 制	意 义
WILL	251	11111011	1. 提供激活选项
WONT	252	11111100	2. 接受请求激活选项
DO	253	11111101	1. 拒绝请求激活选项
DONT	254	11111110	2. 提供禁止选项
			3. 接受禁止请求选项
			1. 同意提供激活选项
			2. 请求激活选项
			1. 不同意提供激活选项
			2. 同意提供禁止选项
			3. 请求禁止选项

一方可以提供激活或禁止一个选项，如果它有权这样做。对方可以同意或不同意这个选项。为了提供激活选项，提供方发送WILL命令，就是说“我能激活这个选项吗？”另一方或者发送DO命令，就是说“同意”；或者发送DONT命令，就是说“我不同意”。为了提供禁止选项，提供方发送WONT命令，就是说“我不再使用这个选项”。回答必须是DONT命令，就是说“我不要再使用这个选项”。

一方可以请求另一方激活或禁一个选项。为了请求激活，请求方发送DO命令，就是说“请激活这个选项”。另一方或者可发送WILL命令，就是说“同意”；或者发送WONT命令，就是说“我不同意”。为了请求禁止，请求方发送DONT命令，就是说“请不要再使用这个选项”。回答必须是WONT命令，就是说“我不再使用它”。

例26.1

图26.4表示了一个选项协商的例。在这个例子中，客户希望服务器将发送给服务器的每一个字符回显。换言之，当在用户终端键盘上键入一个字符时，它传送给服务器，并在被处理之前发送回到用户的屏幕上。回显选项必须被服务器激活，因为正是服务器将这些字符发回到用户的终端。因此，客户应当请求服务器使用DO来激活这个选项。这个请求包括3个字符：IAC，DO和ECHO。服务器接受这个请求，并激活该选项。它通过发送3个字符的认可：IAC，WILL和ECHO通知客户机。

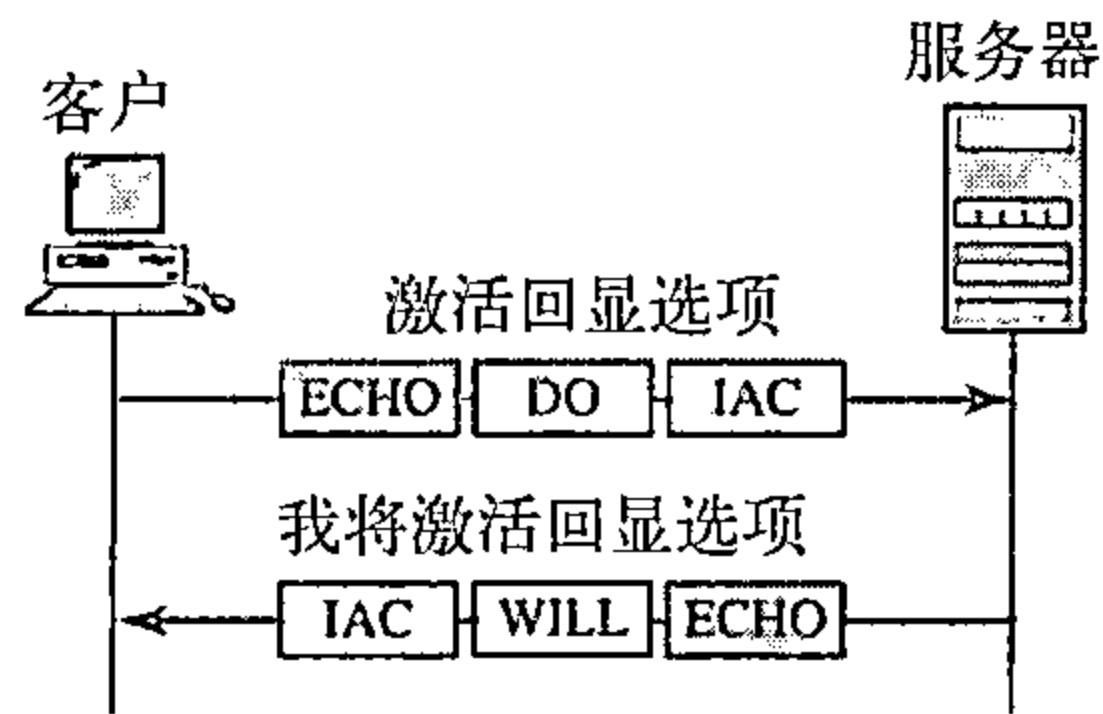


表26.4 子选项协商的NVT字符集

字 符	十进制	二进制	意 义
SE	240	11110000	子选项结束
SB	250	11111010	子选项开始

图26.4 例26.1回显选项

子选项协商 某些选项需要附加的信息。例如，要定义一个终端的类型或速率，协商就要包括一个字符串或数字来定义类型或速率。不论是哪种情况，表26.4所显示的二个选项字符都是进行子选项协商（suboption negotiation）所需要的。

例26.2

图26.5表示了一个子选项协商的例子。在这个例子中，客户希望协商终端的类型。

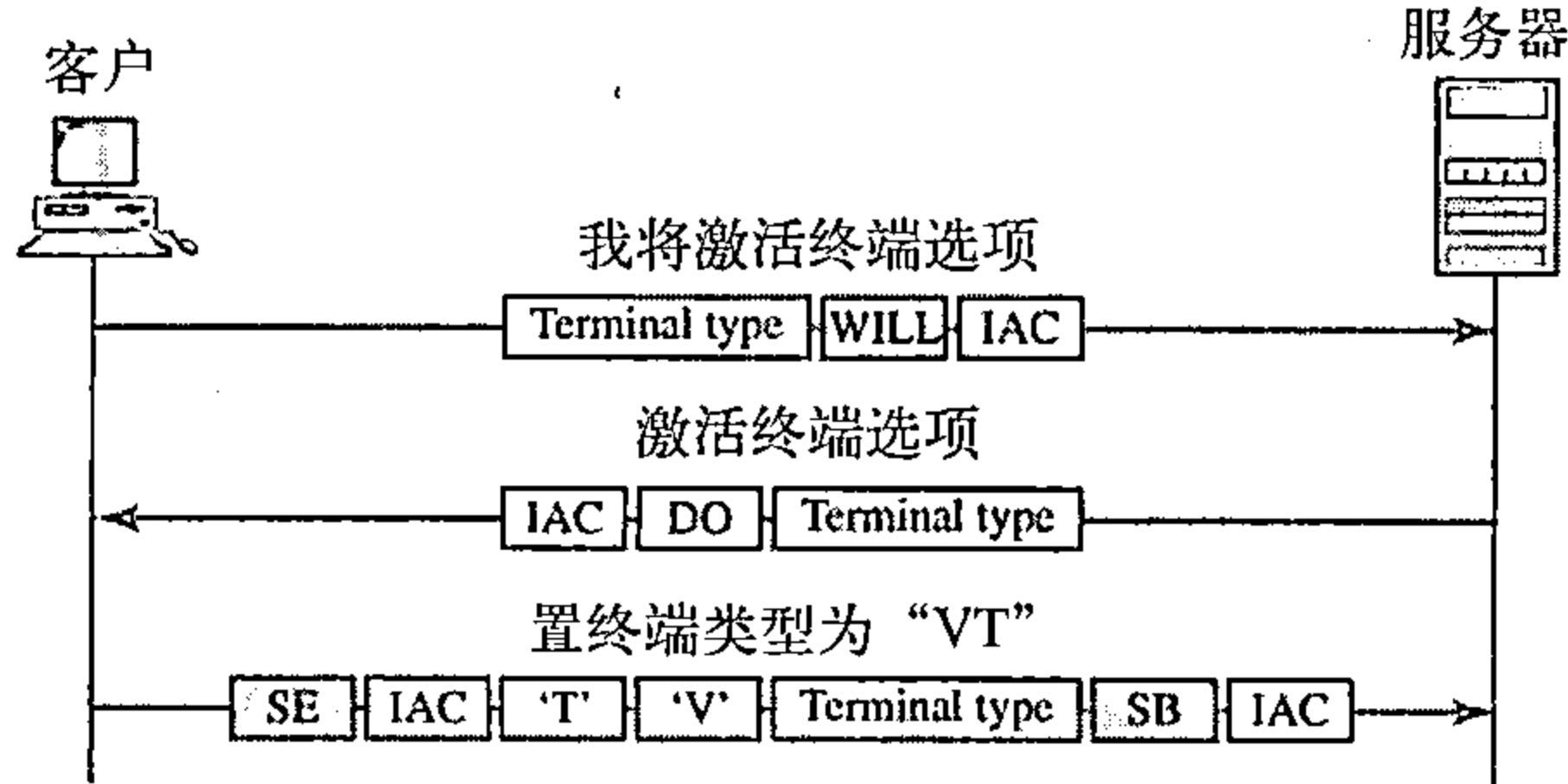


图26.5 子选项协商

操作方式

大多数TELNET实现3种工作方式中的一种：默认方式、字符方式和行方式。

默认方式 如果没有通过选项协商调用其他方式，则使用默认方式（default mode）。在这种方式中，回显由客户完成。用户键入一个字符，客户机将字符回显到屏幕（或打印机）上，但到整个一行完成之前并不发送它。

字符方式 在字符方式（character mode）中，每一个键入的字符从客户机发送给服务器。服务器通常将这个字符回显在客户的屏幕上。如果传输时间较长时（如在一个卫星连接中）这种方式的字符回显可能被延迟时间较长（如在一个卫星连接中）。它还产生网络开销（通信量），因为每一个数据字符必须发送3个TCP段。

行方式 一种新的方式被提出来补偿默认方式的不足。在这种称为行方式（line mode）中，即行编辑（回显、字符擦除和行擦除等等）由客户完成。然后客户将整个行发送给服务器。

26.2 电子邮件

最常用的因特网服务之一是电子邮件 (e-mail)。因特网的设计者们可能从未预料到该应用程序会如此流行。本章讨论电子邮件的几个构件。

早期因特网时期,用电子邮件发送的报文仅是短小的文本文件,它让人们快速地交换便笺。而当今的电子邮件更加复杂,它允许包括文本、音频及频视的报文,同时还允许发送给一个或多个接收者。本章首先讨论电子邮件一般的构架,它包括三个主要的组件:用户代理、报文传输代理和报文访问代理。然后讨论实现这些组件的协议。

26.2.1 构架

为了说明电子邮件的构架,我们给出四种情况。从最简单情况开始逐步增加其复杂性,其中第四种情况是最常用的电子邮件交换。

第一种情况

在第一种情况中,电子邮件的发送方与接收方是在同一个系统内的用户(或相同的应用程序),它们直接地连接到一个共享一个系统中。管理员为每个用户创建一个邮箱,在邮箱中存储接收到的报文。邮箱是本地硬盘的一部分,即批准限定的一个特殊文件。只有邮箱的拥有者可以访问它。如果一个用户Alice要发一个报文给另一个用户Bob时,Alice运行用户代理(UA)程序准备报文并在Bob的邮箱中存储该报文。报文中有发送方与接收方的邮箱地址(文件名)。Bob在他方便时,用用户代理检索他的邮箱并读取其内容。图26.6表明了 this 概念。

这类似于在办公室中职员之间交换传统的便笺。有一个邮件空间,在其中每个职员都有一个用他自己名字命名的邮箱。当Alice要发送一个便笺给Bob时,她写好便笺并将它插入到Bob的邮箱。当Bob检查他的邮箱时,他发现Alice的便笺并读取它。

当电子邮件的发送方和接收方都在同一个系统上,我们仅需要两个用户代理。

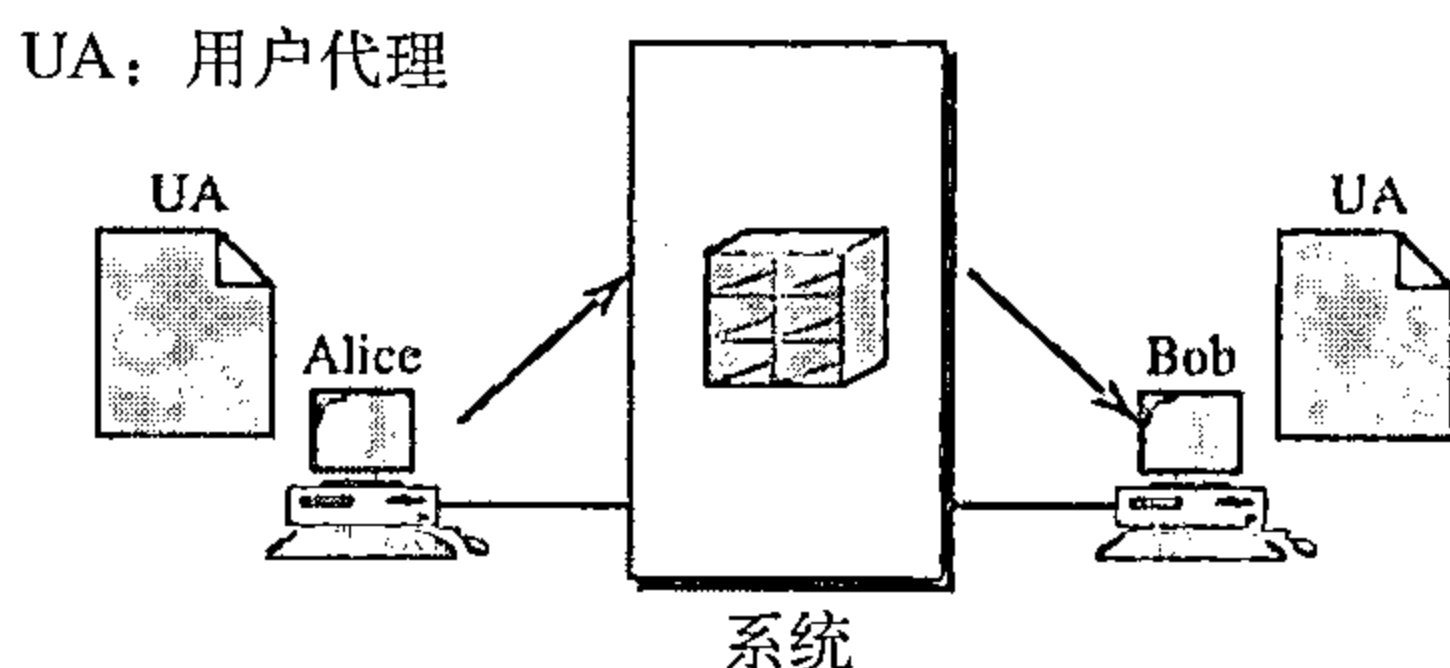


图26.6 电子邮件的第一种情况

第二种情况

在第二种情况中,电子邮件的发送方和接收方是不同系统上两个用户(或应用程序)。此时,我们需要用户代理(user agent, UA)和报文传输代理(message transfer agent, MTA)。如图26.7所示。

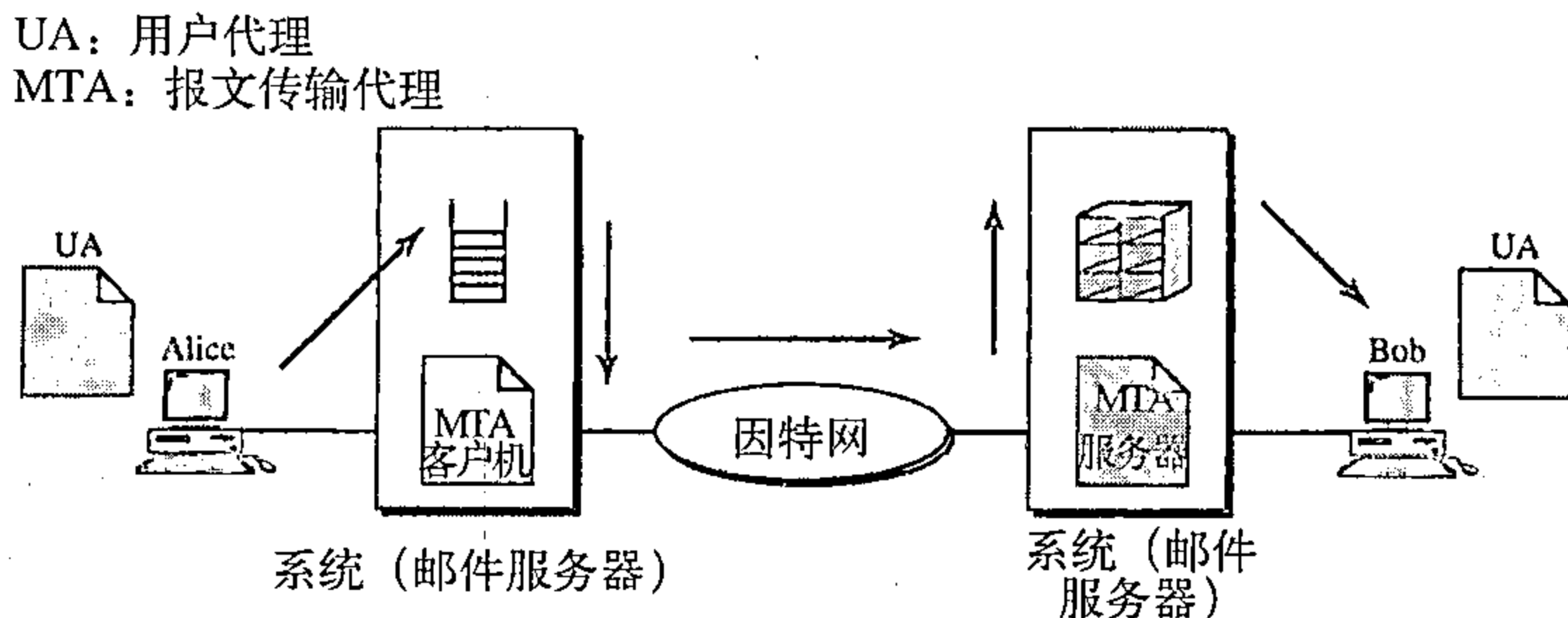


图26.7 电子邮件第二种情况

Alice需要使用用户代理程序发送她的报文到她所在网站的系统。她所在网站的系统(有时称为邮件服务器)使用一个队列存储报文等待发送。Bob也需要一个用户代理程序检索存储

在他的网站邮箱系统中的报文。但是，报文需要通过因特网从Alice的网站发送到Bob的网站。此处需要两个报文传输代理：一个是客户而另一个是服务器。像大多数因特网上的客户/服务器程序一样，服务器程序始终运行，因为它不知道何时用户将会请求一个连接。另一方面，当队列中有发送报文时，可由系统激活客户程序。

当电子邮件的发送方和接收方在不同的系统中时，我们需要两个UA和一对MTA（客户机和服务器）。

第三种情况

在第三种情况中，如同第二种情况一样，Bob直接连接到他的系统，但是Alice没有与她的系统直接连接，Alice或是通过点到点广域网连接到系统，譬如用拨号调制解调器、DSL或有线调制解调器，或者她连接到机构中的一个局域网，该机构用一个邮件服务器处理电子邮件，即所有用户都要将它们的邮件发送到这个邮件服务器上。图26.8表示了这种情况。

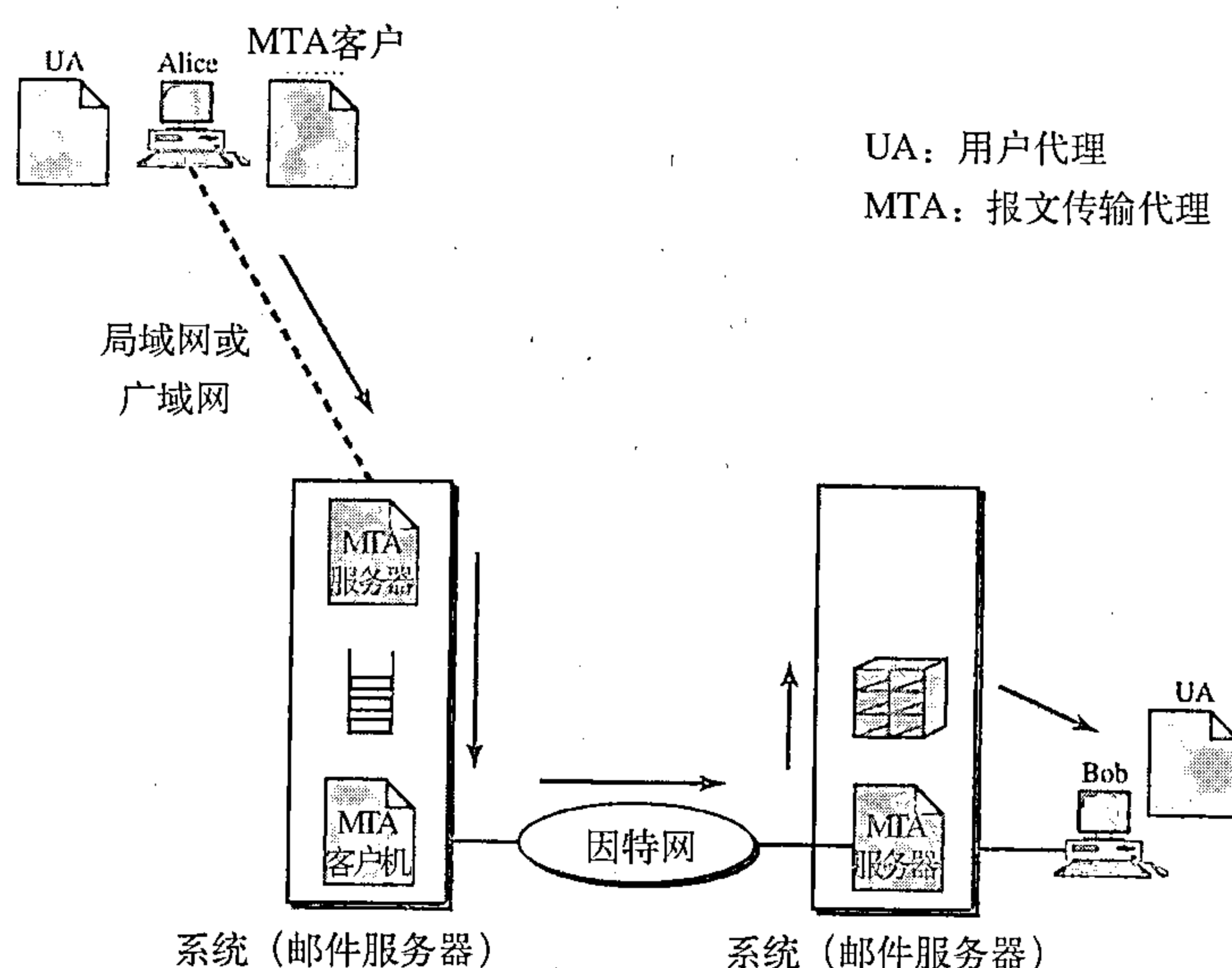


图26.8 电子邮件第三种情况

Alice仍旧需要一个用户代理准备她的报文，然后她需要通过局域网或广域网发送报文。这是通过一对报文传输代理（客户和服务）完成的。每当Alice有报文要发送时，她调用用户代理，并由它调用MTA客户。MTA客户与系统上一直运行的MTA服务器建立一个连接，Alice网站的系统对接收到的所有报文进行排队。然后，它使用MTA客户向Bob网站的系统发送报文，该系统接收报文并存入Bob的邮箱。Bob在他方便时使用他的用户代理进行检索和读取它。注意：我们需要两对MTA客户机/服务器程序。

当发送方通过LAN或WAN连接邮件服务器时，我们需要两对MTA（客户和服务）。

第四种情况

在第四种和最常用的情况中，Bob也是通过广域网或局域网连接到他的邮件服务器。报文到达Bob的邮件服务器后，Bob需要检索它。此时，需要另一组客户机/服务器代理，我们称它们为报文访问代理（message access agent）。Bob使用MAA客户检索他的报文。客户向始终运行的MAA服务器发送一个请求，并请求报文的传输。这情况如图26.9所示。

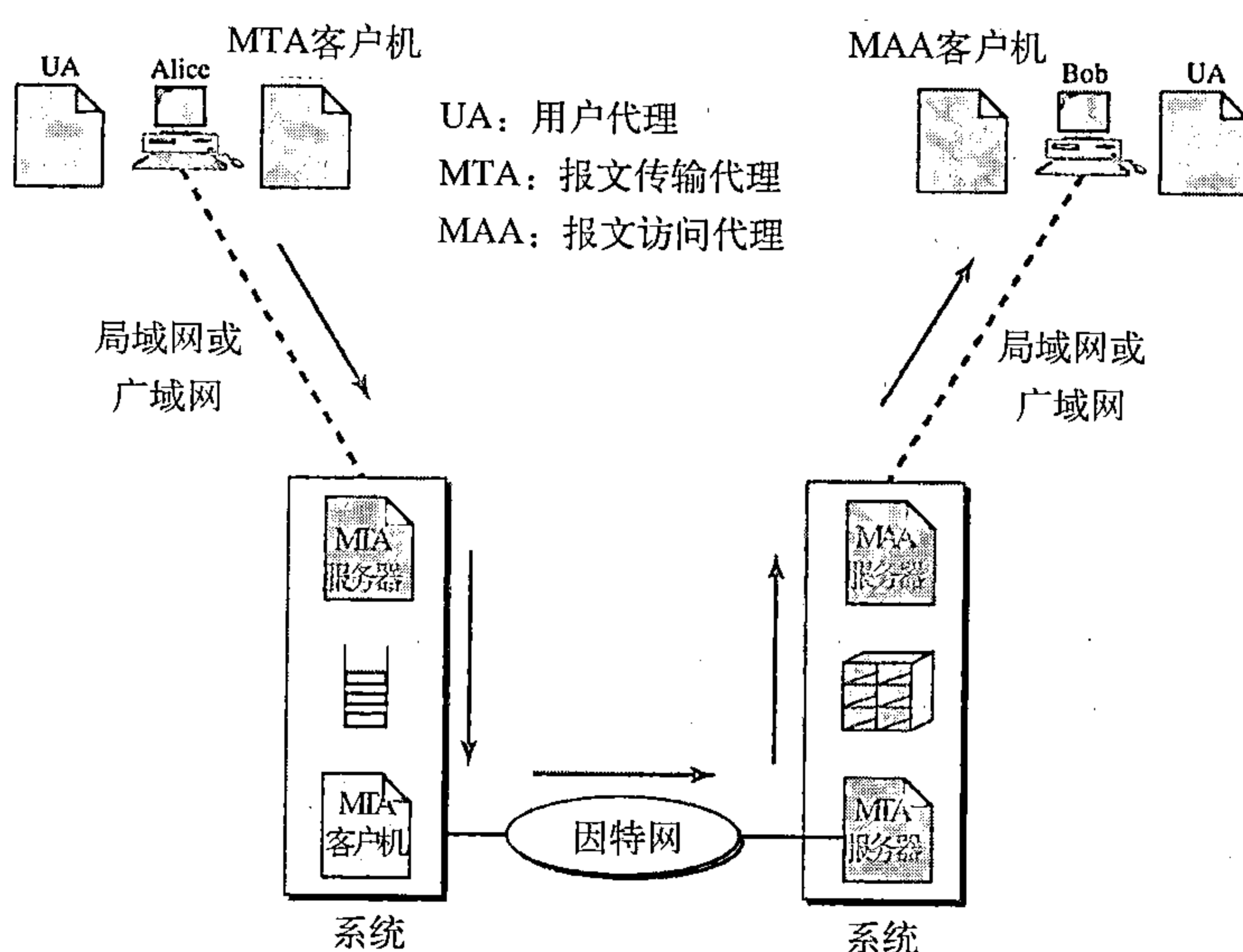


图26.9 电子邮件第四种情况

此处有两个要点。第一点，Bob不能绕开邮件服务器而直接使用MTA服务器。为了直接使用MTA服务器，Bob应该需要始终运行MTA服务器程序，因为它不知道何时报文到达。这意味着如果他通过局域网连接他的系统，Bob必须始终拥有他的计算机。如果他通过广域网连接到他的系统，他必须始终保持连接。今天，这两种情形都不适用。

第二点，Bob需要另一对客户/服务器程序，报文访问程序。因为MTA客户/服务器是一个推（push）程序，客户将报文推入服务器。而Bob需要一个拉（pull）程序。客户需要从服务器拉出报文。图26.10表示了这种差别。

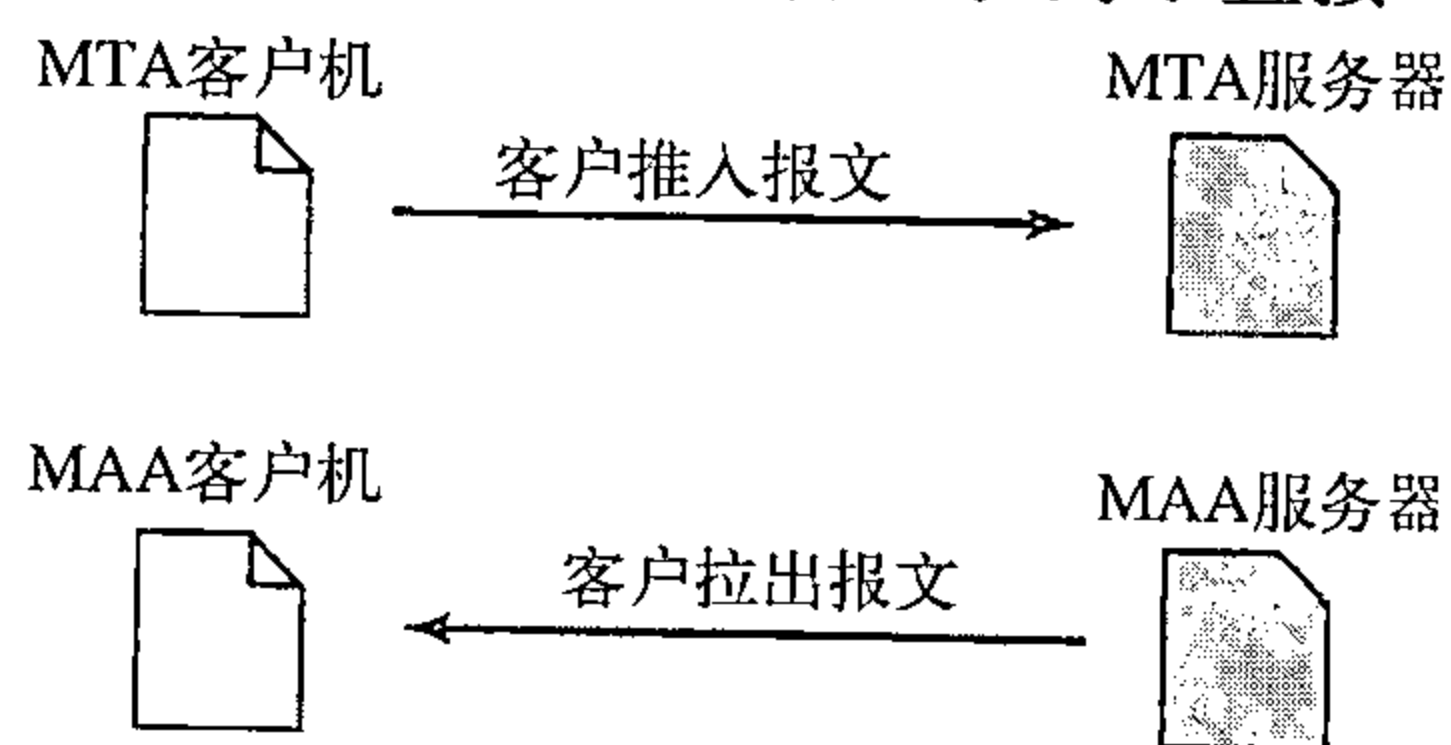


图26.10 电子邮件推和拉

当发送方和接收方通过局域网与广域网连接到邮件服务器时，我们需要两个UA、两对MTA（客户机与服务器）和一对MAA（客户与服务器）。这就是当前最常见的情形。

26.2.2 用户代理

电子邮件系统的第一个组件是用户代理（UA）。它向用户提供服务，使发送和接收一个报文变得更容易。

用户代理提供的服务

用户代理是一个软件包（程序），它由组成、读取、回答和转发报文组成，它也处理邮箱。图26.11表示了一个典型的用户代理的服务。

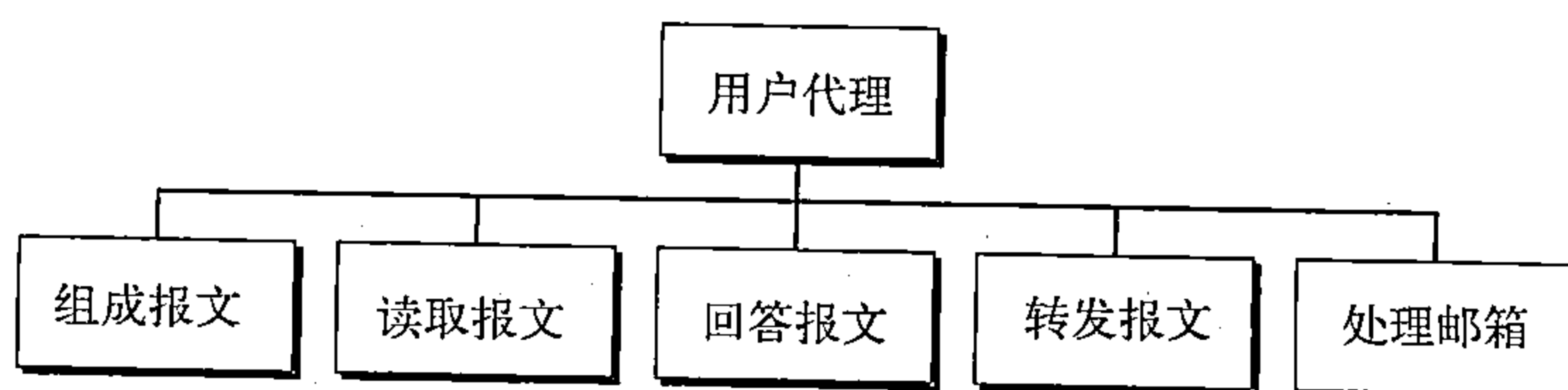


图26.11 用户代理的服务

组成报文 用户代理帮助用户组成发送出去的电子邮件。大多数用户代理在屏幕上提供一个标准的格式由用户来填写，有些甚至还安装了编辑器，可完成拼写检查、文法检查以及其他想要的复杂的字处理任务。当然，用户可以选择使用他喜欢的文本编辑器或字处理器创建报文和输入报文，或剪切和粘贴报文到用户代理模板上。

读取报文 用户代理第二个功能是读取到来的报文。当用户调用用户代理时，它首先在输入邮箱中检查邮件。大多数用户代理都显示每个接收到邮件的一行摘要。每封电子邮件包含下列字段：

1. 编号字段；
2. 表示邮件的状态如新的、已读而没回复或已读并已回复等的标记字段；
3. 报文的长度；
4. 发送方；
5. 可选择的主题字段。

回答报文 读取报文后，用户可用用户代理对报文做出回答。通常用户代理允许对原始的发送方做回复或对该报文的所有接收方做回复。回答的报文可包含原始的报文（用做快速参考）和新的报文。

转发报文 回答是指对发送方或报文副本的接收者发送报文。而转发报文是指向第三方发送报文。用户代理允许接收方附带或不附带额外的注释将报文转发给第三方。

处理邮箱

通常用户代理创建两个邮箱：收件箱和发件箱。每个邮箱是具有一个特殊格式的文件，它能被用户代理所处理。收件箱保留所有接收到的电子邮件直到被用户删除为止。发件箱保留所有已发送的电子邮件直到被用户删除为止。今天，大多数的用户代理有能力创建个性化的邮箱。

用户代理类型

有两种类型的用户代理：命令驱动型和基于GUI型。

命令驱动型 命令驱动型的用户代理属于早期的电子邮件。在服务器中仍然存在这种类型的用户代理。命令驱动型的用户代理通常是从键盘接收单个字符的命令以执行某项任务。例如，用户可以在命令提示行输入字符r回答报文的发送方，或输入R回答发送方和所有的接收者。命令驱动型用户代理的例有：*mail*、*pine*和*elm*。

命令驱动型用户代理的例有*mail*、*pine*和*elm*。

基于GUI型 现在的用户代理都是基于GUI型。它们包含图形用户接口（GUI）组件，该组件允许用户使用键盘和鼠标与软件进行交互。它们还有一些图形组件，如图标、菜单条和使服务更容易访问的窗口。基于GUI的用户代理有Eudora、微软的Outlook和Netscape。

基于GUI的用户代理有Eudora、Outlook和Netscape。

发送邮件

为了发送邮件，用户通过UA创建邮件，邮件看起来很像邮政邮件，它有一个信封和一个报文（见图26.12所示）。

信封 信封（envelope）通常包含发信人的地址和收件人的地址。

报文 报文包含头部（header）和主体（body）。报文头部定义了发信人、收件人、报文的主题及其他信息（如编码类型）。报文的主体包含了由收信人读取的真正信息。

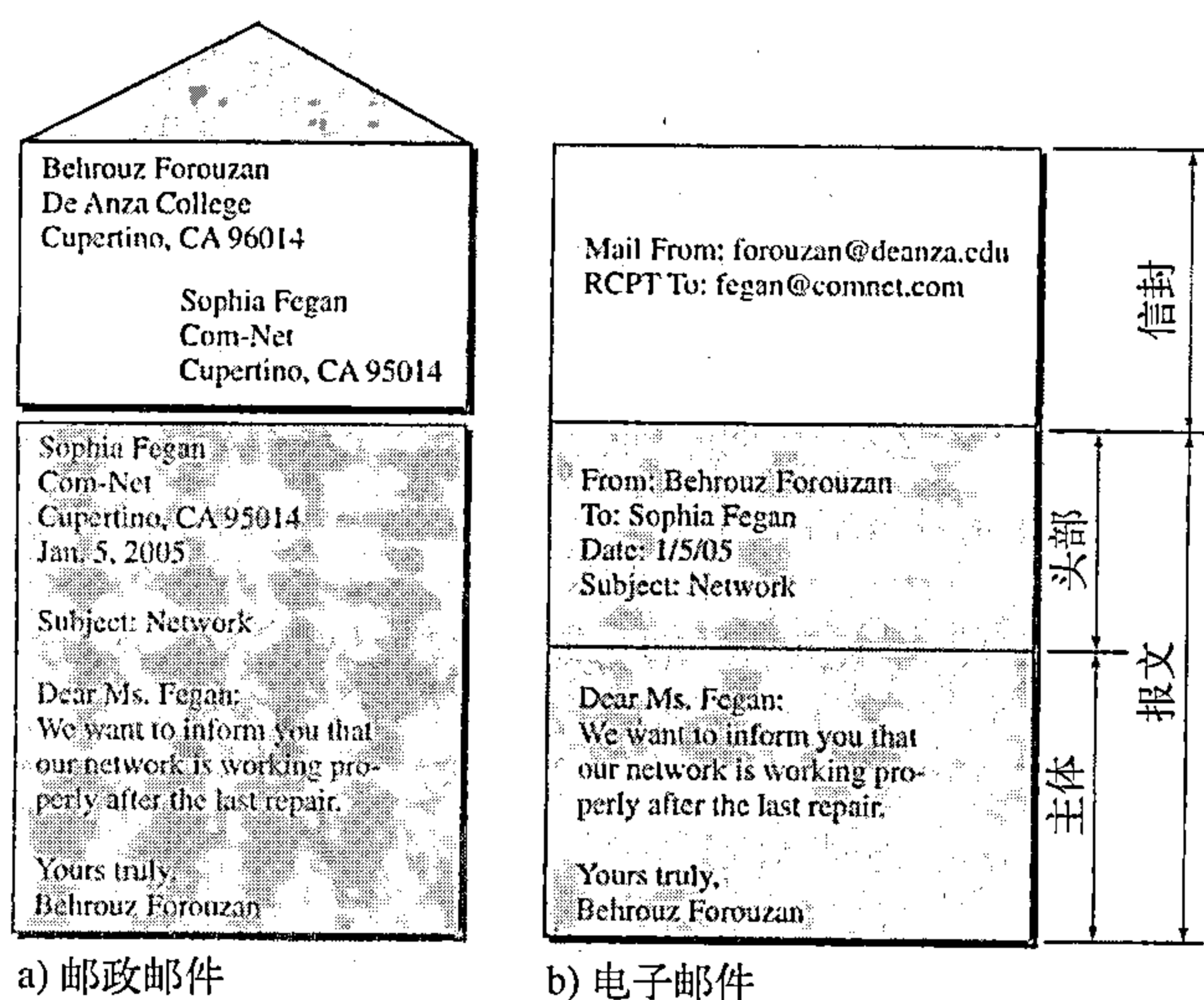


图26.12 电子邮件的格式

接收邮件

用户（或定时器）触发用户代理检查邮箱。如果用户有邮件，UA就用一个通知来告诉用户。如果用户准备读取邮件，则会显示一个列表，其中列表的每一行包含了邮箱中关于一个特定报文的信息概要。这个概要通常包括发信人的邮件地址、主题以及发送和接收此邮件的时间。用户可以选择任何一个报文，在屏幕上显示它的内容。

地址

为了传递邮件，邮件处理系统必须使用具有唯一地址的寻址系统。在因特网中，地址由两部分构成：本地部分（local part）和域名（domain name），并且用符号@分隔开（见图26.13所示）。

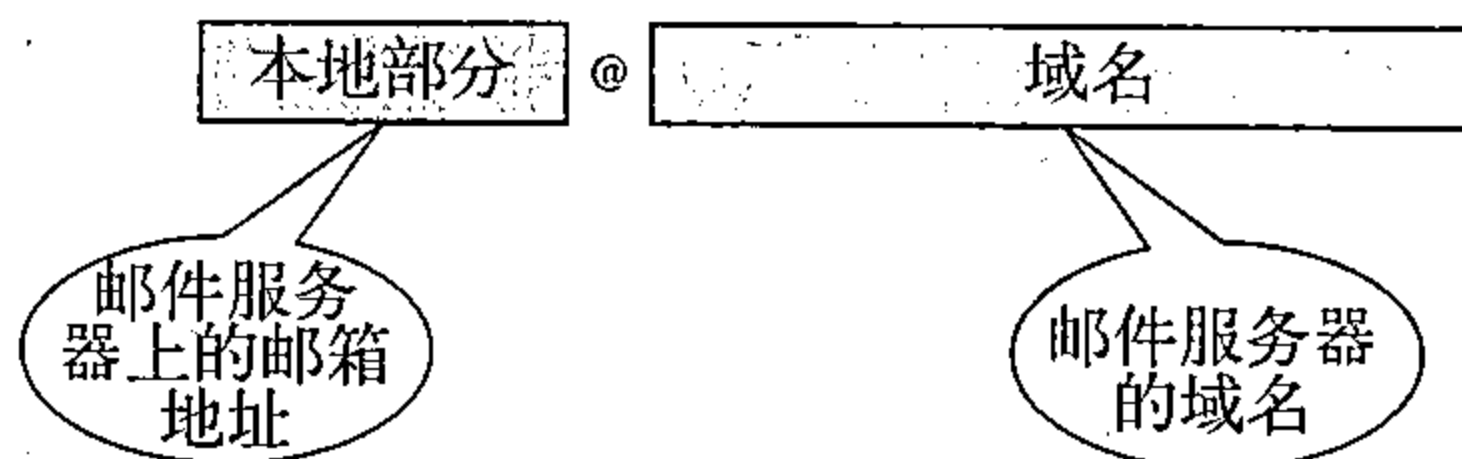


图26.13 电子邮件地址

本地部分 本地部分定义一个特定文件的名称，它称为用户邮箱，在用户邮箱中存储了用户所有接收到的邮件，以便用户代理进行读取。

域名 地址的第二部分是域名。一个组织机构通常选择一个或者多个主机用于接收和发送邮件；这些主机有时称为邮件服务器或交换器。分配给每一个邮件交换器的域名或者是来自DNS数据库，或者是一个逻辑名字（例如，该组织机构的名字）。

邮件列表

电子邮件允许用一个名字，即别名（alias），来表示多个不同的电子邮件地址，这称为邮件列表。每当发送一个报文时，系统就将收信人的名字与别名数据库进行比较；如果所定义的别名有一个邮件列表，那么就将报文分开，每一个对应列表中的一项，然后交给MTA处理。如果别名没有邮件列表，那么该名字就是接收地址，并将一个单独的报文传递给邮件传输实体。

MIME

电子邮件有一个简单的结构。但是，它的简单是有代价的。它只能发送使用NVT 7位ASCII格式的报文。换言之，它有一些限制。例如，它不能使用7位ASCII字符支持的语言（如法文、德文、希伯利文、俄文、中文以及日文）。另外，它不能用来发送二进制文件或音

频或视频数据。

多用途因特网邮件扩充 (Multipurpose Internet Mail Extension, MIME) 是一个辅助协议, 它允许非ASCII数据能够通过电子邮件传送。MIME在发送方将非ASCII数据转换成NVT ASCII数据, 并将其传递给MTA客户机通过因特网发送出去。在接收方再转换到原来的数据。

可以将MIME想像为一组软件功能, 它能够将非ASCII数据转换成ASCII数据, 以及进行相反的转换。如图26.14所示。

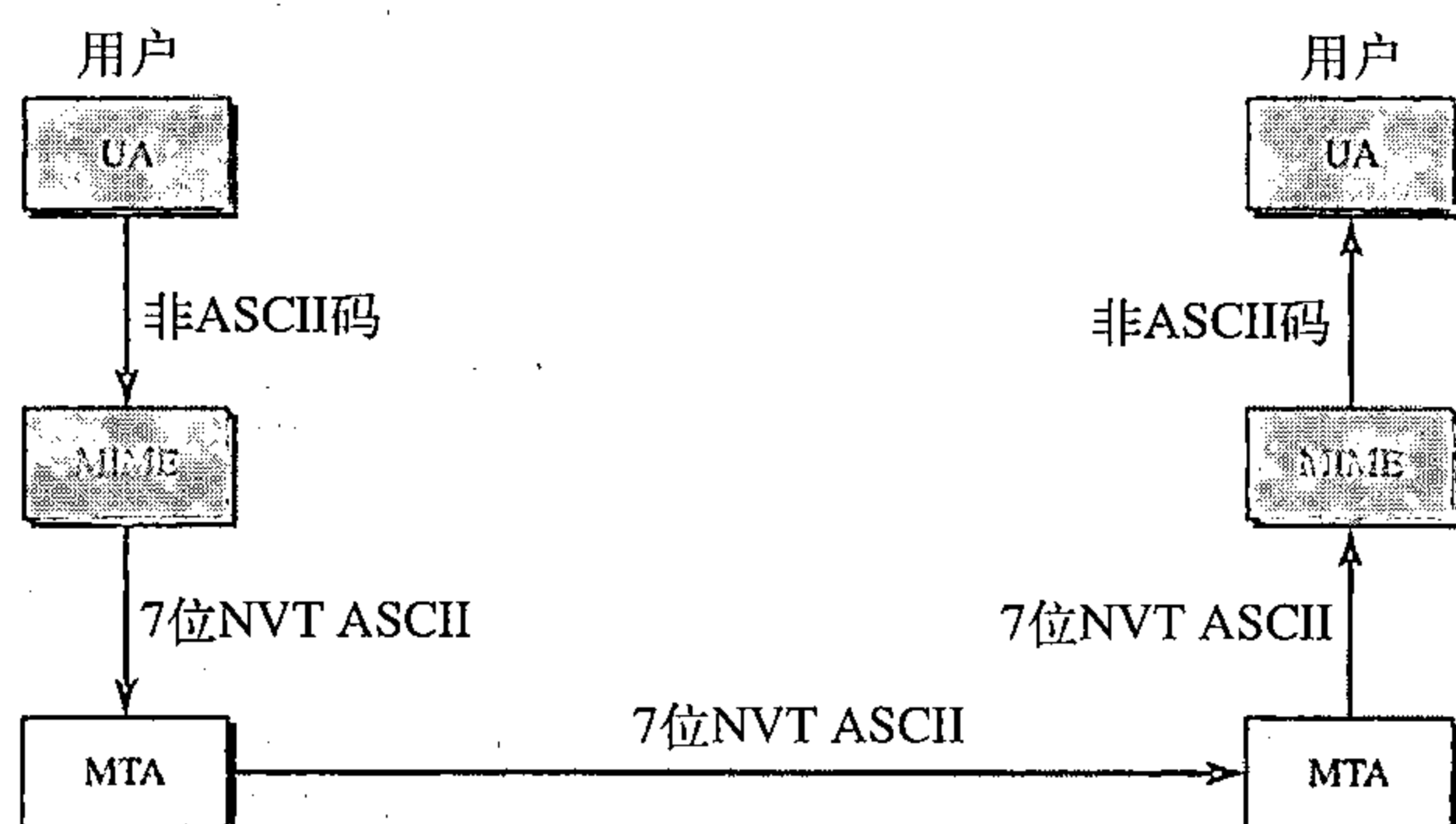


图26.14 MIME

MIME定义了5种头部, 将MIME的头加在原来电子邮件的头部以定义转换参数:

1. MIME版本;
2. 内容-类型;
3. 内容-传送-编码;
4. 内容-标识符;
5. 内容-描述。

图26.15表示了MIME的头部, 我们将详细讨论每一个头部。

MIME版本 这个头部定义MIME使用的版本, 当前版本是1.1。

MIME-版本: 1-1

内容-类型 这个头部定义报文主体使用的数据类型, 内容类型和内容子类型用一个斜杠分隔开。根据子类型的不同, 头部还可包含其他一些参数。

内容-类型: <类型/子类型; 参数>

MIME允许7种不同的数据类型, 这些类型在表26.5中列出。

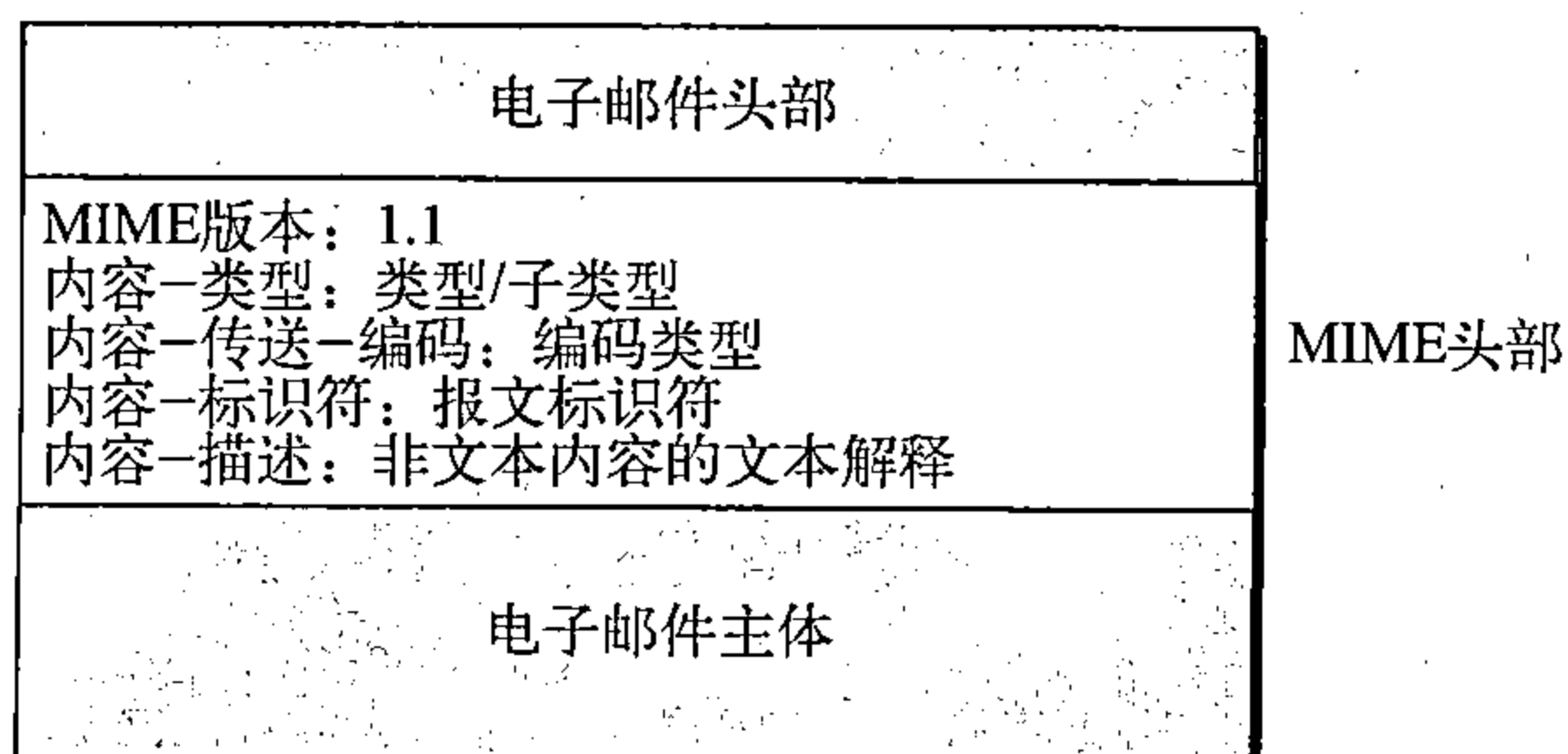


图26.15 MIME头部

内容-传送-编码 为了传送的目的这个头部定义将报文编码为一些0和一些1的方法:

Content-Transfer-Encoding: <type> 内容-传送-编码: <类型>

表26.6列出了5种类型的编码。

表26.5 MIME中的数据类型和子类型

类 型	子 类 型	说 明
正文	普通 HTML	无格式 HTML格式（见第27章）
多部分	混合 并行 摘要 可供选择的	主体包含不同数据类型的有序部分同上， 但无序 与混合子类型相似，但默认的是报文/RFC822 几个部分是同一个报文的的不同版本
报文	RFC822 部分 外部主体	主体是一个封装的报文 主体是一个较大报文的一部分 主体是另一个报文的参照
图像	JPEG GIF	JPEG格式的图像 GIF格式的图像
视频	MPEG	MPEG格式的视频信号
音频	基本	8kHz的单声道话音编码
应用	PostScript 8位组流	Adobe PostScript 一般的二进制数据（8位字节）

表26.6 内容-传送-编码

类 型	说 明
7位	NVT ASCII字符和短行
8位	非ASCII字符和短行
二进制	长度不限行的ASCII字符
Base64	6位数据块被编码成8位ASCII字符
引用中可打印的	非ASCII字符被编码成等号后面跟随一个ASCII码

内容-标识符 这个头部在多报文环境中唯一地标识整个报文。

内容标识符：id=<内容标识符>

内容-描述 这个头部定义主体是否为图像、音频或视频。

内容-描述：<描述>

26.2.3 报文传输代理：SMTP

实际的邮件传输是由报文传输代理（MTA）完成的。为了发送邮件，系统必须有客户MTA，而为了接收邮件，系统必须有服务器MTA。在因特网中，定义MTA客户机和服务器的形式化协议称为简单邮件传输协议（Simple Mail Transfer Protocol, SMTP）。如以前所述，在大多数情况下（第四种情况），使用两对客户/服务器MTA程序。图26.16表示了这种情况中，SMTP管辖的范围。

SMTP在发送方与接收方邮件服务器之间以及两个邮件服务器之间使用两次。稍后我们将看到，在邮件服务器与接收方之间还需要另一个协议。

SMTP只定义了如何来回发送命令和响应。每一个网络有权选择一种实现的软件包。在本节剩下的部分，我们讨论SMTP的邮件传输机制。

命令和响应

SMTP使用一些命令和响应在MTA客户和MTA服务器之间传输报文（见图26.17所示）。

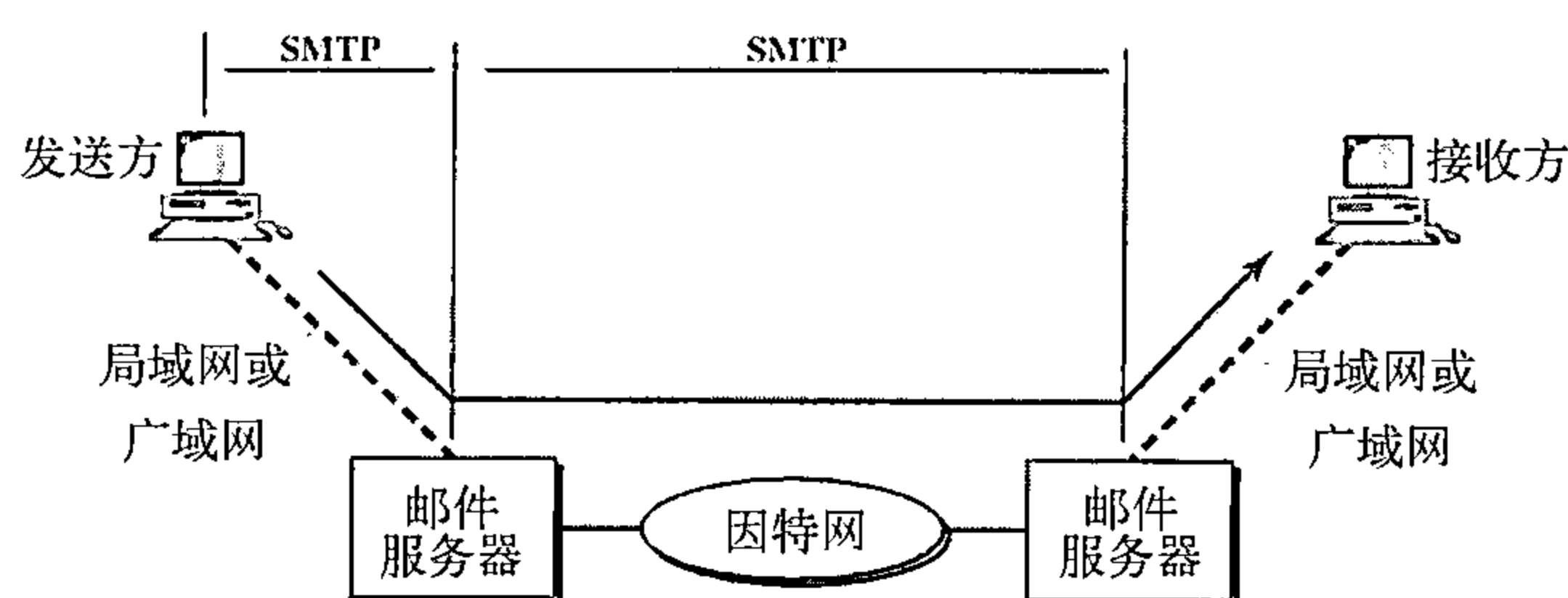


图26.16 SMTP管辖的范围

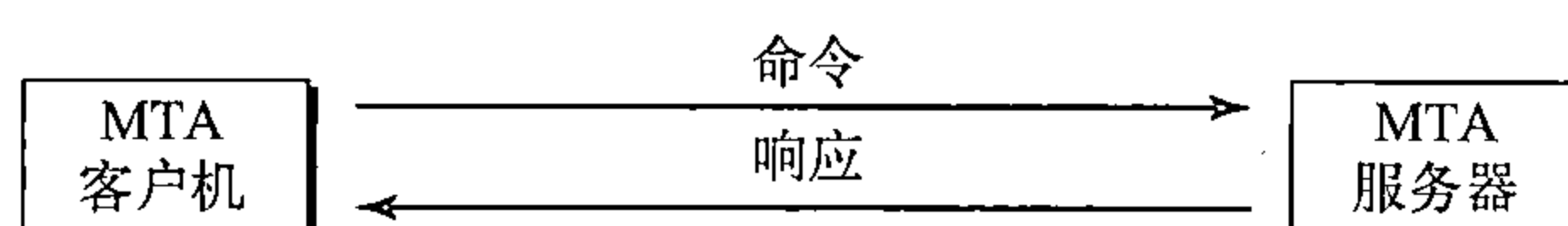


图26.17 命令和响应

每一命令或响应都以一个二字符（回车和换行）的行结束标记来终止。

命令 命令是从客户发送给服务器，命令的格式如图26.18所示。它包括一个关键词，后跟着0个或多个变量。SMTP定义了14个命令，前5个是强制性的，每一种实现都必须支持这5个命令。后面的3个是常用的，并且是着重推荐的。最后的6个很少使用。

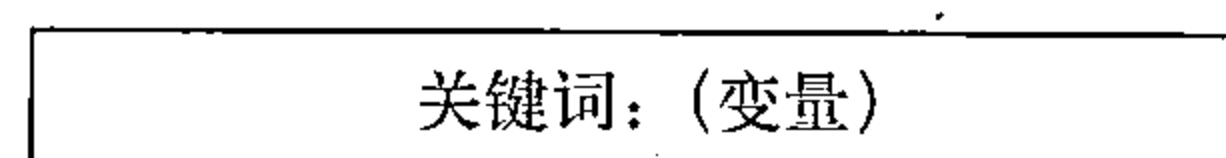


图26.18 命令格式

在表26.7中列出各种命令。

表26.7 命令

关键词	变量	关键词	变量
HELO	发送方的主机名	NOOP	
MAIL FROM	发信人	TURN	
RCPT TO	预期的收信人	EXPN	需要扩展的邮件列表
DATA	邮件主体	HELP	命令名
QUIT		SEND FROM	预期的收信人
RSET		SMOL FROM	预期的收信人
VERFY	需要验证收信人的名字	SMAL TROM	预期的收信人

响应 响应是服务器发送给客户。响应是一个3位数字码，后面可以跟着附加的文本信息。表26.8中列出了各种响应。

如表所示，响应划分为四大类。代码的最高位数字（2，3，4和5）定义大类。

邮件传输阶段

传输一个邮件共有三个阶段：连接建立、邮件传输和连接终止。

例26.3

让我们观察如何直接使用SMTP发送电子邮件和如何模拟本节所描述命令与响应。我们使用TELNET登录到端口25（SMTP的熟知端口号），然后用命令直接发送电子邮件。在这个例子中，forouzanb@adelphia.net发送一封电子邮件给他自己。前面的三行表示TELNET试图与Adelphia邮箱服务器建立连接。

连接建立后，我们输入SMTP命令，然后接收响应，如下所示。用黑色表示命令，而用彩色表示响应。注意：为了更清晰起见，我们增加了用一些“=”符号指定的注释行。这些行

不是电子邮件过程的部分。

表26.8 响应

代 码	说 明	代 码	说 明
	肯定的完成答复	452	命令异常中止，存储空间不足
211	系统状态或帮助回答		永久性否定的完成答复
214	帮助报文	500	语法错误，命令无法识别
220	服务就绪	501	参数或变量中有语法错误
221	服务关闭传输通道	502	命令未执行
250	请求命令完成	503	命令序列不正确
251	用户不是本地的，报文将被转发	504	命令暂时未执行
	肯定的中间答复	550	命令未执行，邮箱不可用
354	开始邮件输入	551	用户不是本地的
	瞬态否定的完成答复	552	所请求的动作异常停止，超出存储位置
421	服务不可用	553	所请求动作未发生，不允许使用邮箱名
450	邮箱不可用	554	事务失败
451	命令异常中止，本地错误		

```
$ telnet mail.adelphia.net 25
Trying 68.168.78.100 ...
Connected to mail.adelphia.net (68.168.78.100).
```

```
===== Connection Establishment =====
220 mta13.adelphia.net SMTP server ready Fri, 6 Aug 2004 ...
HELO mail.adelphia.net
250 mta13.adelphia.net

===== Mail Transfer =====
MAIL FROM: forouzanb@adelphia.net
250 Sender <forouzanb@adelphia.net> Ok
RCPT TO: forouzanb@adelphia.net
250 Recipient <forouzanb@adelphia.net> Ok
DATA
354 Ok Send data ending with <CRLF>.<CRLF>
From: Forouzan
TO: Forouzan

This is a test message
to show SMTP in action.
.
===== Connection Termination =====
250 Message received: adelphia.net@mail.adelphia.net
QUIT
221 mta13.adelphia.net SMTP server closing connection
Connection closed by foreign host.
```

26.2.4 报文访问代理：POP和IMAP

邮件传递的第一阶段和第二阶段使用SMTP。但是，因为SMTP是一个推（push）协议，第三阶段不使用SMTP。它将报文从客户推入服务器。换言之，大量数据（报文）的方向是从客户到服务器。另一方面，第三阶段需要一个拉（pull）协议，客户机必须从服务器拉出报文。大量数据的方向是从服务器到客户。因此，第三阶段使用报文访问代理协议。

目前有两种报文访问代理协议：邮局协议版本3（POP3）和因特网邮件访问协议版本4（IMAP4）。图26.19显示了在大多数情况下（第四种情况），这两种协议的位置。

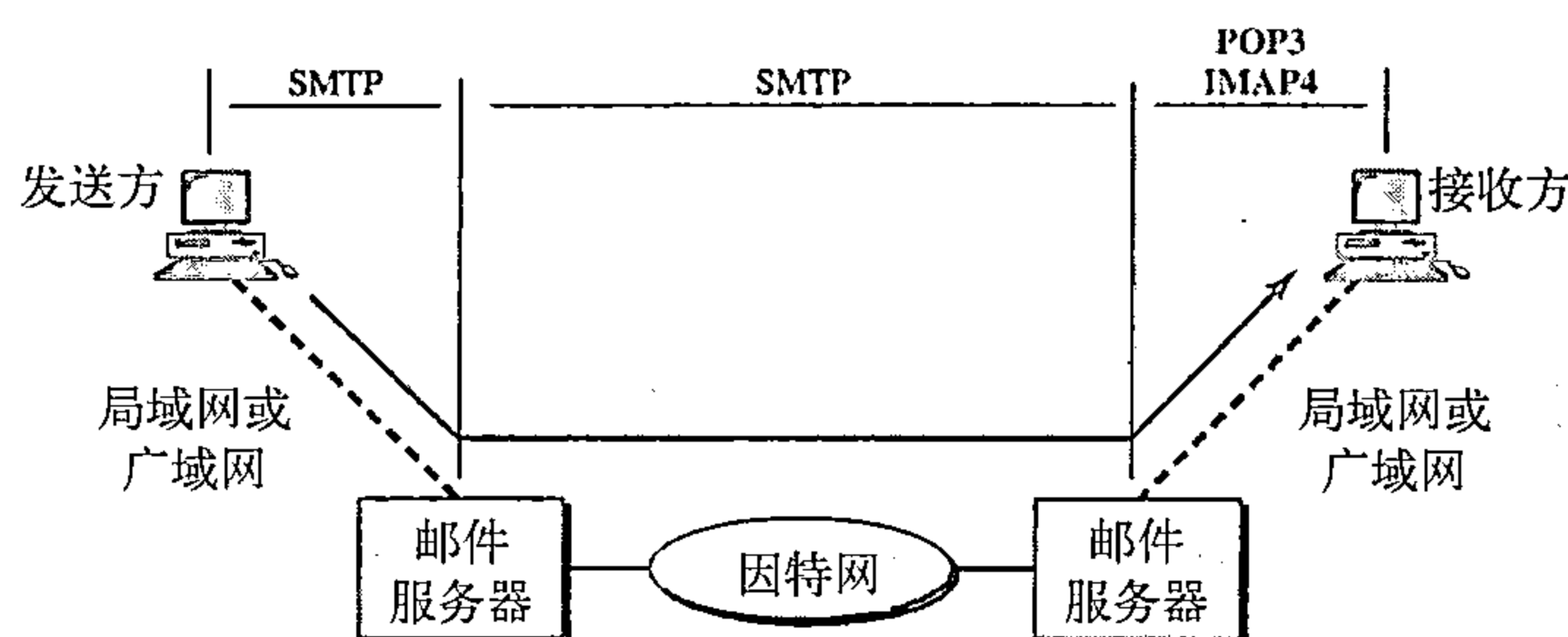


图26.19 POP3和IMAP

POP3

邮局协议版本3 (Post Office Protocol version 3, POP3) 比较简单,但是在功能上受到一定的限制。客户端POP3软件安装在收信人的计算机中,服务器POP3软件安装在邮件服务器中。

用户需要从邮件服务器的邮箱中下载邮件时,客户端发起邮件访问操作。客户端(用户代理)开启与使用服务器110端口之间的TCP连接。然后发送用户名和口令来访问邮箱。用户就可以逐条列出和读取邮件信息了。图26.20说明了一个使用POP3下载邮件的例子。

POP3有两种模式:删除模式和保存模式。在删除模式中,当邮件从邮箱中读取以后就会从邮箱中删除该邮件。在保存模式中,邮件经过读取以后仍然保存在邮箱中。当用户在固定的计算机上工作时,能够在阅读和回复邮件后保存并组织接收到的邮件,此时通常使用删除模式。当用户远离他的主计算机(如在便携机上)访问邮件时,通常应该使用保存模式。此时可以阅读邮件,但是邮件通常仍然会保存在系统中以备日后读取和组织。

IMAP4

另外一个邮件访问协议是因特网邮件访问协议版本4 (Internet Mail Access Protocol version 4, IMAP4)。IMAP4与POP3相似,但有更多特点:功能更强并更复杂。

POP3在某些方面存在缺点,它不允许用户在服务器上组织邮件;用户在服务器上不能有不同文件夹。(当然,用户能够在自己的计算机上建立文件夹。)同时,POP3不允许用户在下载邮件之前部分地查看邮件的内容。

IMAP4提供下列额外的功能:

- 用户在下载电子邮件之前,可以检查电子邮件头部。
- 用户在下载电子邮件之前,可以读取电子邮件内容中的特定字符串。
- 用户可以部分地下载电子邮件。这在带宽受限制而电子邮件中包含了需要高带宽的多媒体信息时特别有用。
- 用户可以在邮件服务器上创建或删除邮箱,也可以改变邮箱的名字。
- 用户可以在文件夹中创建邮箱的层次结构以用于邮件存储。

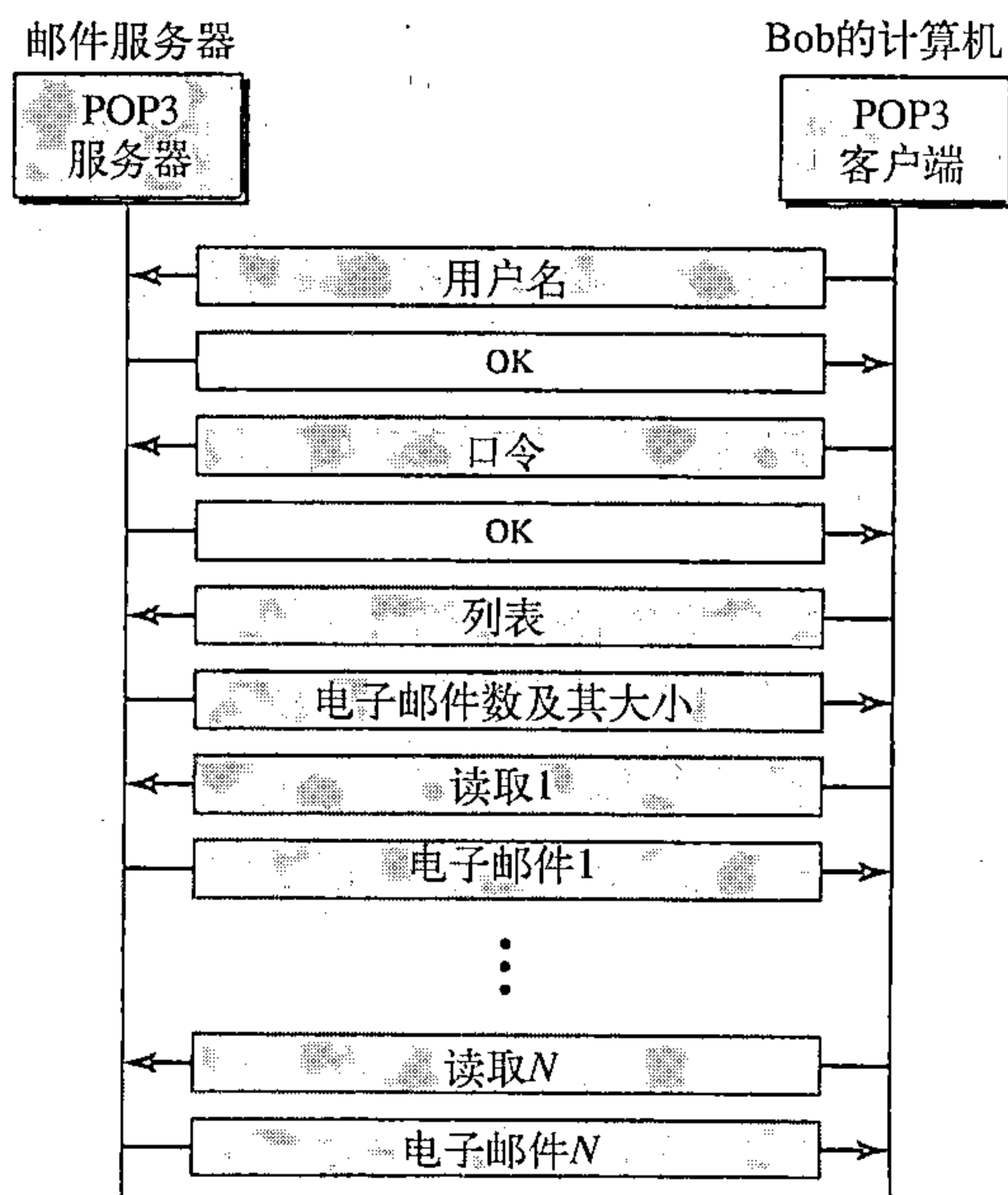


图26.20 POP3中命令与响应的交换

26.2.5 基于Web的邮件

电子邮件是一种很常见的应用，目前有一些Web网站对任何访问者都提供服务。两个常用的网站是Hotmail和Yahoo。其思想很简单，Alice的邮件通过HTTP（见第27章）从她的浏览器传输到她的邮件服务器，报文从邮件发送服务器通过SMTP发送到邮件接收服务器。最后，该报文从邮件接收服务器（Web服务器）通过HTTP发送到Bob的浏览器。

最后一个阶段值得注意一下，通常使用HTTP代替POP3或者IMAP4。当Bob需要读取他的邮件时，他会发送报文到Web网站上（例如Hotmail）。Web网站会返回一个表单，由Bob填写，包括登录名和口令。如果登录名和口令匹配，电子邮件会从Web服务器以HTML格式传送到Bob的浏览器。

26.3 文件传输

从一个计算机向另一个计算机传输文件是在联网或互联网环境中最常见的任务之一。事实上，今天在因特网上数据交换的最大量应属于文件传输。在本节中，我们讨论传送文件最常用的协议：文件传输协议（FTP）。

26.3.1 文件传输协议（FTP）

文件传输协议（File Transfer Protocol, FTP）是TCP/IP提供的标准机制，用于从一个主机将文件复制到另一个主机。虽然从一个系统到另一系统传送文件看起来是很简单而且直观，但首先还要解决一些问题。例如。两个系统可能使用不同的文件名约定。两个系统使用不同的方式来表示文本和数据。两个系统具有不同的目录结构。所有这些问题都已经由FTP以一种非常简单巧妙的方法解决了。

FTP与其他客户/服务器应用程序的不同之处在于它在主机之间建立两个连接。一个连接用于数据传输，另一个用于控制信息传输（命令和响应）。将命令和数据传输分开使得FTP的效率更高。控制连接使用非常简单的通信规则。我们需要传输的只是一次一行命令或者一行响应。另一方面，数据传输需要更加复杂的规则，因为传输的数据类型种类多。

FTP使用两个熟知TCP端口：端口21用于控制连接，端口20用于数据连接。

FTP使用TCP服务。它需要两个TCP连接。熟知端口21用于控制连接，而熟知端口20用于数据连接。

图26.21说明了FTP的基本模型。客户有三个组件：用户接口、客户控制进程和客户数据传输进程。服务器由两个组件：服务器控制进程和服务器数据传输进程。控制连接是在控制进程之间进行的，而数据连接是在数据传输进程之间进行的。

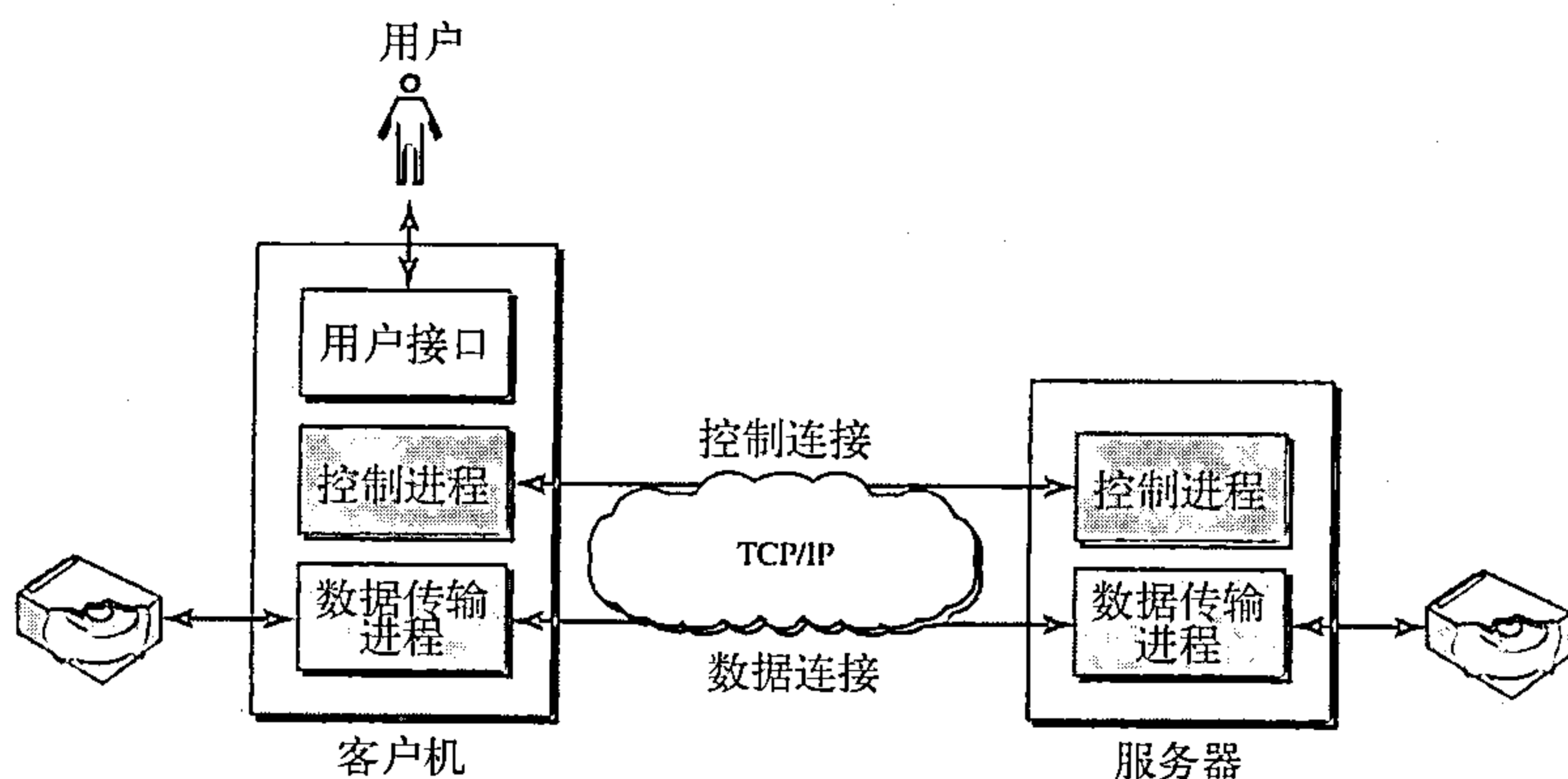


图26.21 FTP

在整个交互的FTP会话期间，控制连接（controlconnection）始终处于连接状态。数据连接（data control）则在每次传输文件时开启然后关闭。每当牵涉到文件传输的命令被使用时，数据连接就被打开，而当文件传输完毕时连接就关闭。换言之，当用户开始FTP会话时，控制连接就被打开。在控制连接处于打开状态期间，如果传输多个文件，那么数据连接可以打开和关闭多次。

通过控制连接的通信

FTP使用如STMP那样的方法通过控制连接进行通信。它使用7位ASCII字符集（见图26.22所示）。通信是通过命令和响应来完成的。这种简单方法对控制连接是合适的。因为我们一次发送一条命令（或响应）。每一条命令或响应都是一个短行，因此不必担心它的文件格式或文件结构，每一行结束处是两个字符（回车和换行）的行结束记号。

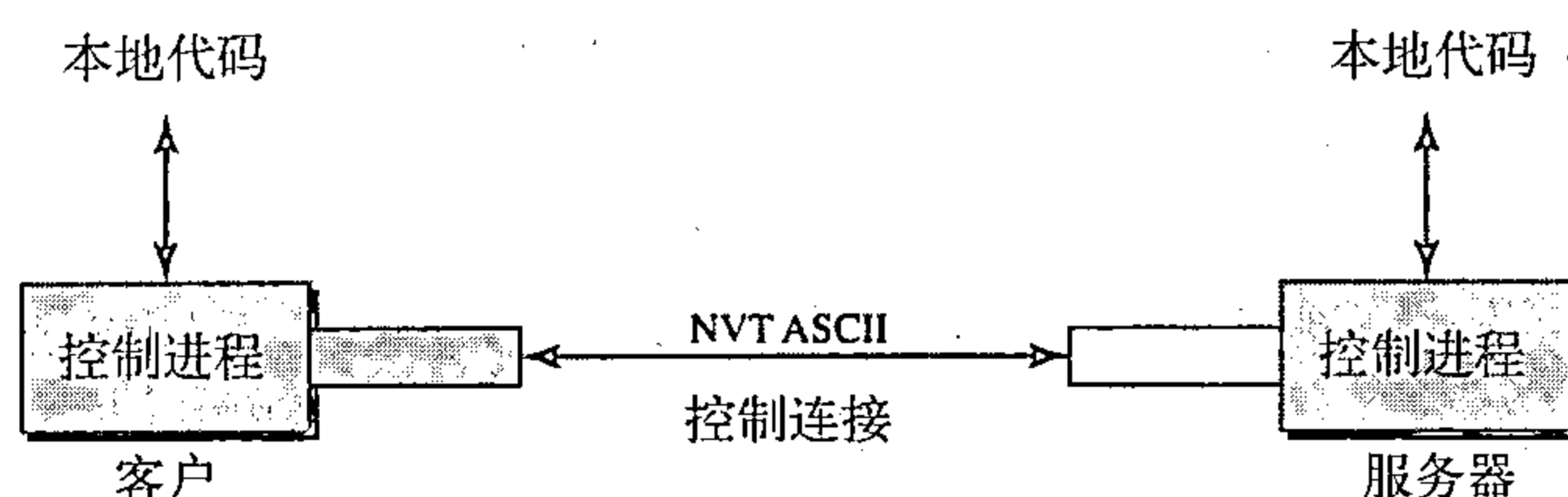


图26.22 使用控制连接

通过数据连接的通信

数据连接的目的和实现与控制连接是不同的。我们通过数据连接来传输文件。在控制连接上发送的命令控制下，在数据连接上进行文件传输。但是，我们要记住，FTP的文件传输表示三件事之一：

- 从服务器将一个文件复制到客户。这称为读取文件，这是在RETR命令监管下完成的；
- 从客户将一个文件复制到服务器。这称为存储文件，这是在STOR命令监管下完成的；
- 从服务器向客户发送目录或文件名列表。这是在LIST命令监管下完成的。应注意是FTP将目录或文件名列表当作一个文件，它在数据连接上发送。

客户必须定义要传送的文件类型、数据结构和传输方式。在通过数据连接传送数据之前，我们通过控制连接准备传输。异构性问题可以通过定义三个通信属性来解决：文件类型、数据结构和传输方式（见图26.23所示）。

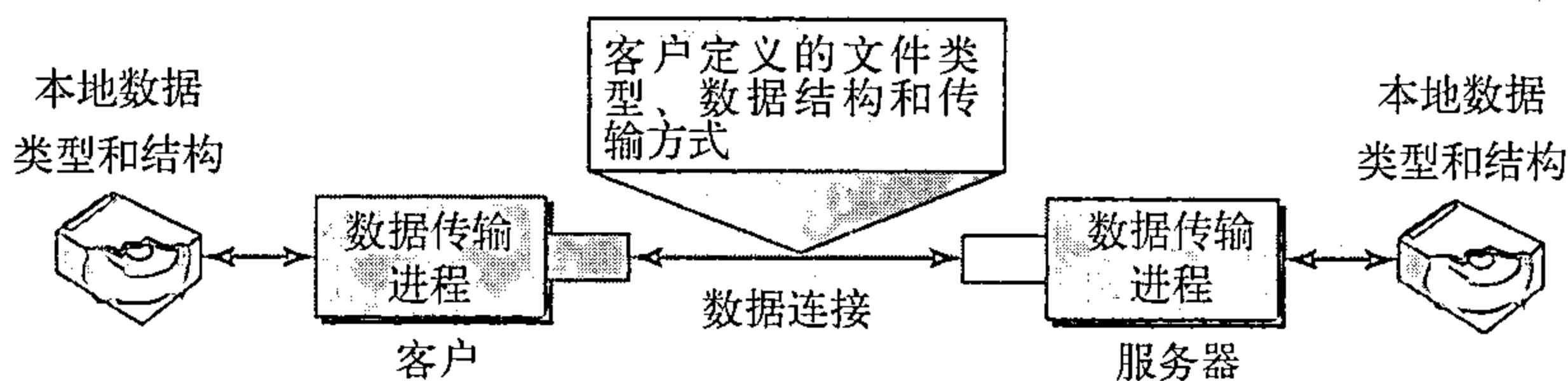


图26.23 使用数据连接

文件类型 FTP能够在数据连接上传送下列文件中的一种：ASCII文件、EBCDIC文件和图像文件。ASCII文件（ASCII file）是传送文本文件的默认格式，每个字符使用7位ASCII进行编码。发送方将文件从它自己的表示方式转换成ASCII字符，而接收方从ASCII字符转换成它自己的表示方式。如果连接的一方或两方使用EBCDIC编码（IBM使用的传输格式），则可用EBCDIC编码传送文件。图像文件（image file）是传送二进制文件的默认格式。这种文件是作为连续的位流传送而没有解释或编码。它在大多情况下是用来传送二进制文件，如已编

译的程序。

数据结构 FTP可以使用下列数据结构中的一种在数据连接上传送文件：文件结构、记录结构和页面结构。在**文件结构**（file structure）中，文件是连续的字节流。在**记录结构**（record structure）中，文件划分成一些记录，这只能用于文本文件。在**页面结构**（page structure）中，文件划分成一些页面，每一个页面有一个页面号和页面头部，页面可以随机地或顺序地存储或访问。

传输方式 FTP可以使用下列三种传输方式之一在数据连接上传送文件：流方式、块方式和压缩方式。**流方式**（stream mode）是默认方式，数据作为连续的字节流从FTP传递给TCP，TCP负责将数据划分成适当大小的段。如果数据是简单的字节流（文件结构），就不需要文件结束符。在这种情况下，发送方使用文件结束符关闭数据连接。如果数据划分为记录（记录结构），则每一个记录将有一个1字节的记录结束（EOR）字符，而在文件的结束处有一个文件结束（EOF）字符。在**块方式**（block mode）中，数据可以按块从FTP传递给TCP。在这种情况下，每一个块的前面有一个3字节的头部。第一个字节称为块描述符，下面两个字节定义块的大小，以字节为单位。在**压缩方式**（compressed mode）中，如果文件很大，数据可进行压缩。通常使用的压缩方法是游程长度编码。在这种方法中，数据单元连续出现可用一个出现和重复数来代替。在文本文件中，这通常是空格。在二进制文件中，空格字符常常被压缩。

例26.4

下面给出了使FTP会话读出目录中的项目清单。彩色的行表示来自服务器控制连接的响应，黑色行表示用户发送的命令。黑色背景中带白色的行表示数据传输。

```
$ ftp voyager.deanza.fhda.edu
Connected to voyager.deanza.fhda.edu.
220 (vsFTPd 1.2.1)
530 Please login with USER and PASS.
Name (voyager.deanza.fhda.edu:forouzan): forouzan
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls reports
227 Entering Passive Mode (153,18,17,11,238,169)
150 Here comes the directory listing.
```

drwxr-xr-x	2	3027	400	10976	Sep 24	2002	business
drwxr-xr-x	2	3027	400	10976	Sep 24	2002	personal
drwxr-xr-x	2	3027	400	10976	Sep 24	2002	school

```
226 Directory send OK.
ftp> quit
221 Goodbye.
```

1. 在创建了控制连接后，FTP服务器在控制连接上发送220（服务器就绪）响应。
2. 客户发送它的名字。
3. 服务器用331（用户名正确，需要口令）响应。
4. 客户发送口令（不显示出来）。
5. 服务器用230（用户登录正确）响应。
6. 客户发送列表命令（ls报告）去寻找报告名目录中的列表。
7. 现在服务器用150响应，并打开数据连接。
8. 然后服务器在数据连接上发送文件列表或目录（作为一个文件）。当整个列表（文件）发送出后，服务器在控制连接上用226（关闭连接）响应。

9. 客户现在有两个选择。它可以使用QUIT命令请求关闭这个控制连接，或者它可以发送另一个命令开始另一个活动（甚至打开另一个数据连接）。在我们的例子中，客户发送一个QUIT命令。

10. 接收到QUIT命令后，服务器用221（服务关闭）响应，然后关闭控制连接。

26.3.2 匿名FTP

要使用FTP，用户就需要在远程服务器上有一个账号（用户名）和口令。有些网站有一组可供公众使用的文件可激活匿名FTP（anonymous FTP）。要使用这些文件，用户不需要有账号和口令。因此可用anonymous这个字作为用户名和使用guest这个字作为口令。

用户访问这样的系统是受限制的。有些网站只允许用户使用命令的一个子集。例如，大多数网站允许用户复制某些文件，但不允许用户任意查找目录。

例26.5

我们给出了匿名FTP的例子，假定在internic.net上有些公众数据是可用的。

```
$ ftp internic.net
Connected to internic.net
220 Server ready
Name: anonymous
331 Guest login OK, send "guest" as password
Password: guest
ftp > pwd
257 '/' is current directory
ftp > ls
200 OK
150 Opening ASCII mode
```

```
lrm
```

```
ftp > close
221 Goodbye
ftp > quit
```

推荐读物

为了深入讨论与本章有关的主题，推荐阅读下列读物和网站，在本书末列出方括号[...]中的参考资料。

读物

远程登录在[For06]的第18章和[Ste94]的第26章中有讨论。电子邮件在[For06]第20章、[PD03]的9.2节、[Com04]的第32章、[Tan03]的7.2节以及[Ste94]的第28章中有讨论。FTP在[For06]的第19章、[Ste94]的第27章和[Com04]的第34章中有讨论。

网站

下面网站与本章的主题有关。

- www.ietf.org/rfc.html有关请求评论的信息。

请求评论

下列请求评论是有关于TELNET:

137, 340, 393, 426, 435, 452, 466, 495, 513, 529, 562, 595, 596, 599, 669, 679, 701, 702, 703, 728, 764, 782, 818, 854, 855, 1184, 1205, 2355

下列请求评论是有关于SMTP、POP和IMAP:

196, 221, 224, 278, 524, 539, 753, 772, 780, 806, 821, 934, 974, 1047, 1081, 1082, 1225, 1460, 1496, 1426, 1427, 1652, 1653, 1711, 1725, 1734, 1740, 1741, 1767, 1869, 1870, 2045, 2046, 2047, 2048, 2177, 2180, 2192, 2193, 2221, 2342, 2359, 2449, 2683, 2503

下列请求评论是有关于FTP:

114, 133, 141, 163, 171, 172, 238, 242, 250, 256, 264, 269, 281, 291, 354, 385, 412, 414, 418, 430, 438, 448, 463, 468, 478, 486, 505, 506, 542, 553, 624, 630, 640, 691, 765, 913, 959, 1635, 1785, 2228, 2577

DNS是在[AL98]、[For06]的第17章、[PD03]的9.1节和[Tan03]的7.1中有讨论。

本章小结

- TELNET是一种客户/服务器应用程序, 它使用户能够登录到远程机器上, 使用该用户能够访问到远程系统。
- TELNET使用网络虚拟终端(NVT)系统在本地系统上对字符进行编码。在服务器机器上, NVT将字符解码为远程机器可接受的形式。
- NVT使用数据字符集和控制字符集。
- 在TELNET中, 控制字符嵌入在数据流中, 前面加上一个解释为控制(IAC)的控制字符。
- 选项的特点是可以增强TELNET进程。
- TELNET在使用服务之前或之中可以用协商方法在客户和服务器之间设置传输条件。
- TELNET的实现可以工作在默认方式、字符方式和行方式。
 1. 在默认方式中, 客户向服务器一次发送一行;
 2. 在字符方式中, 客户向服务器一次发送一个字符;
 3. 在行方式中, 客户向服务器一次发送一行。
- SMTP、POP3和IMAP4等多个程序支持因特网上的电子邮件服务。
- 在电子邮件中, UA准备报文、创建信封并将报文放入信封中。
- 在电子邮件中, 邮件地址包括两部分: 本地部分(用户邮箱)和域名, 其形式是localpart@domainname。
- 在电子邮件中, 多用途因特网邮件扩充(MIME)允许传送多个报文。
- 在电子邮件中, MTA在因特网、局域网或广域网上传输邮件。
- SMTP使用命令和响应在MTA客户和MTA服务器之间传输报文。
- 传输邮件报文的步骤是:
 1. 连接建立;
 2. 邮件传输;
 3. 连接终止。
- 邮局协议版本3(POP3)和因特网邮件访问协议版本4(IMAP4)都是用于从邮件服务器拉出报文的协议。
- 文件传输协议(FTP)是因特网中用于文件传输的一种程序。
- FTP需要两个用于数据传输的连接: 一个控制连接和一个数据连接。
- 在两个不同的系统之间FTP使用NVT ASCII进行通信。

- 在真正传输文件之前，客户通过控制连接定义文件类型、数据结构和传输方式。
- 响应是在连接建立期间，由服务器向客户发送的。
- 文件传输的类型有三种：
 1. 从服务器将一个文件复制到客户；
 2. 从客户将一个文件复制到服务器；
 3. 从服务器向客户机发送目录列表或文件名。
- 匿名FTP提供了一个方法，使一般公众能使用远程网站的文件。

习题

复习题

1. 在TELNET中，本地登录和远程登录有什么不同？
2. 在NVT中，控制字符与数据字符如何区别？
3. 在TELNET中，如何进行选项协商？
4. 试描述SMTP所使用的寻址系统。
5. 在电子邮件中，用户代理的任务是什么？
6. 在电子邮件中，MIME是什么？
7. 为什么电子邮件需要POP3或IMAP4？
8. FTP的目的是什么？
9. 试描述FTP两种连接的功能。
10. FTP可以传输的文件类型有哪几种？
11. 三种FTP传输方式是什么？
12. 存储文件与检索文件有什么不同？
13. 什么是匿名FTP？

练习题

14. 试给出从客户TELNET对11110011 00111100 11111111进行二进制传输时所发送的位序列。
15. 如果TELNET使用字符方式，使用命令cp file1 file2将一个名为file1的文件复制成名为file2的文件，在客户和服务端之间要往返多少个字符？
16. 要完成练习15的任务，在TCP级需要发送的最少的位个数是多少？
17. 要完成练习15的任务，在数据链路层这一级（使用以太网）需要发送的最少的位个数是多少？
18. 在练习17中，有用的位与总的位之比是多少？
19. 试解释一个TELNET客户或服务端接收到的下列字符序列（以十六进制表示）：

a. FF FB 01	b. FF FE 01
c. FF F4	d. FF F9
20. 一个发信方发送无格式文本，试给出MIME头部。
21. 一个发信方发送JPEG文本，试给出MIME头部。
22. 如果TCP已建立一个连接，为什么传送邮件还要建立一个连接？
23. 为什么对匿名FTP会有限制？一个不讲道德的用户可能会怎样做？
24. 试解释为什么FTP没有一个报文格式。

探究题

25. 试给出从默认方式切换到字符方式时，在TELNET客户和服务端之间交换的字符序列。
26. 试给出从默认方式切换到行方式时，在TELNET客户和服务端之间交换的字符序列。
27. 试给出在SMTP中，从aaa@xxx.com到bbb@yyy.com的连接建立阶段。
28. 试给出在SMTP中，从aaa@xxx.com到bbb@yyy.com的报文传送阶段。报文是“Good morning my friend.”
29. 试给出在SMTP中，从aaa@xxx.com到bbb@yyy.com的连接终止阶段。
30. 如果正在传输数据时突然其控制连接出现严重故障，你认为会发生什么问题？
31. 试进行一些研究，找出TELNET提供的一些扩展选项。
32. 另一个登录协议是Rlogin，试比较TELNET与Rlogin。
33. UNIX中有一个称为安全壳（SSH）的更安全的登录协议，试对该协议进行一些研究。

第27章 万维网与超文本传输协议

万维网（World Wide Web, WWW）是分布在世界各地互相链接在一起的信息仓库。WWW具有独特的灵活性、可移植性以及友好的用户界面，它与因特网上提供的其他服务都不同。WWW项目最初是由CERN（欧洲粒子物理研究所）发起，用来创建一个能处理为科学研究用的分布式资源的系统。在本章中，我们首先讨论与Web相关的问题，然后讨论用来从Web中检索信息的协议HTTP。

27.1 体系架构

现在，WWW是一个分布式的客户/服务器服务，在这种方式下，客户机用浏览器能够使用服务器提供的服务。然而，提供的服务是分布在许多称为站点的位置上。如图27.1所示。

每一个站点拥有一个或多个文档，也就是所谓的Web页面。每一个Web页面可以包含一个本地站点或其他站点的页面链接。这些页面可以通过浏览器找到并浏览。让我们详细观察图27.1所示的情形。客户端需要查找它知道在站点A上的一些信息，它用浏览器（一个用来获取Web文档的应用程序）发送请求。请求中还包含了站点的地址和称为URL的Web页面信息，关于URL稍后会对其进行讨论。在站点A的服务器找到文档并将它发送给客户机。当用户浏览文档时，她发现有关其他文档的一些参考，包括在站点B上的一个Web页面。这个参考有新站点的URL，用户也有兴趣看这个文档。客户向新的站点发送另一个请求，这样就得到了新的页面。

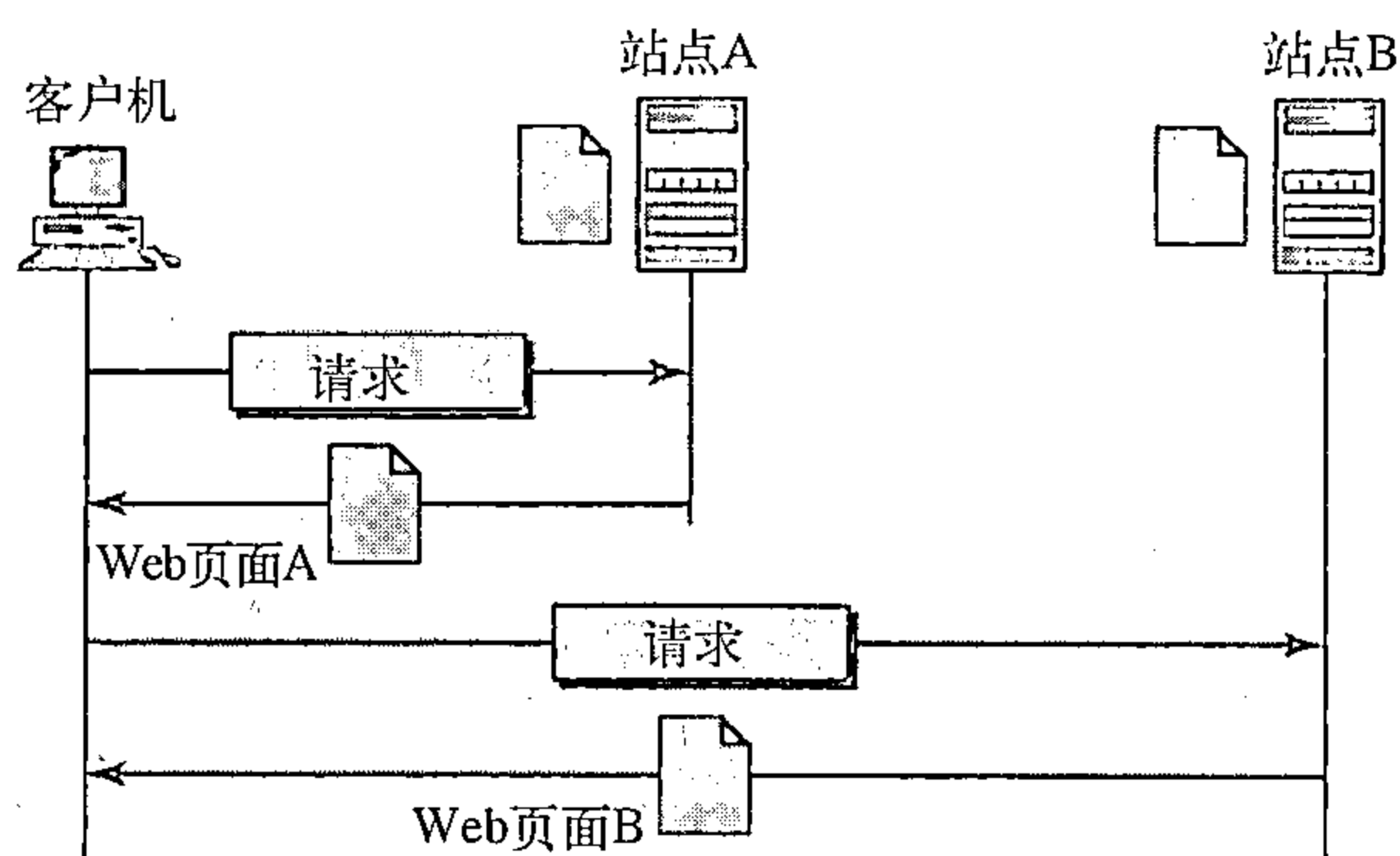


图27.1 WWW的体系架构

我们详细观察图27.1所示的情形。客户端需要查找它知道在站点A上的一些信息，它用浏览器（一个用来获取Web文档的应用程序）发送请求。请求中还包含了站点的地址和称为URL的Web页面信息，关于URL稍后会对其进行讨论。在站点A的服务器找到文档并将它发送给客户机。当用户浏览文档时，她发现有关其他文档的一些参考，包括在站点B上的一个Web页面。这个参考有新站点的URL，用户也有兴趣看这个文档。客户向新的站点发送另一个请求，这样就得到了新的页面。

27.1.1 客户（浏览器）

许多开发商提供了商用的浏览器来解释和显示Web文档，而所有这些浏览器几乎都使用相同的体系架构。每一种浏览器（browser）通常由三部分构成：一个控制程序、客户协议和一些解释程序。控制程序从键盘或者鼠标接收输入，并使用客户端程序访问文档。获取文档以后，控制程序使用解释程序将文档显示在屏幕上。客户机协议可以是前面所述的协议之一，如FTP或者HTTP（在本章后面会进行讨论）。解释程序可以是HTML、Java或者JavaScript，这取决于文档的类型。在本章后部分，我们将基于文档的类型讨论如何使用这些解释程序（见图27.2所示）。

27.1.2 服务器

Web页面存储在服务器中。每次当客户机请求到达时，相应的文档就发送给客户。为了提高效率，服务器通常把请求的文件存储在内存的高速缓存中，访问内存比访问硬盘要快。服务器也可以通过多线程、多处理器提高效率。在这种情况下，服务器可以一次应答更多的请求。

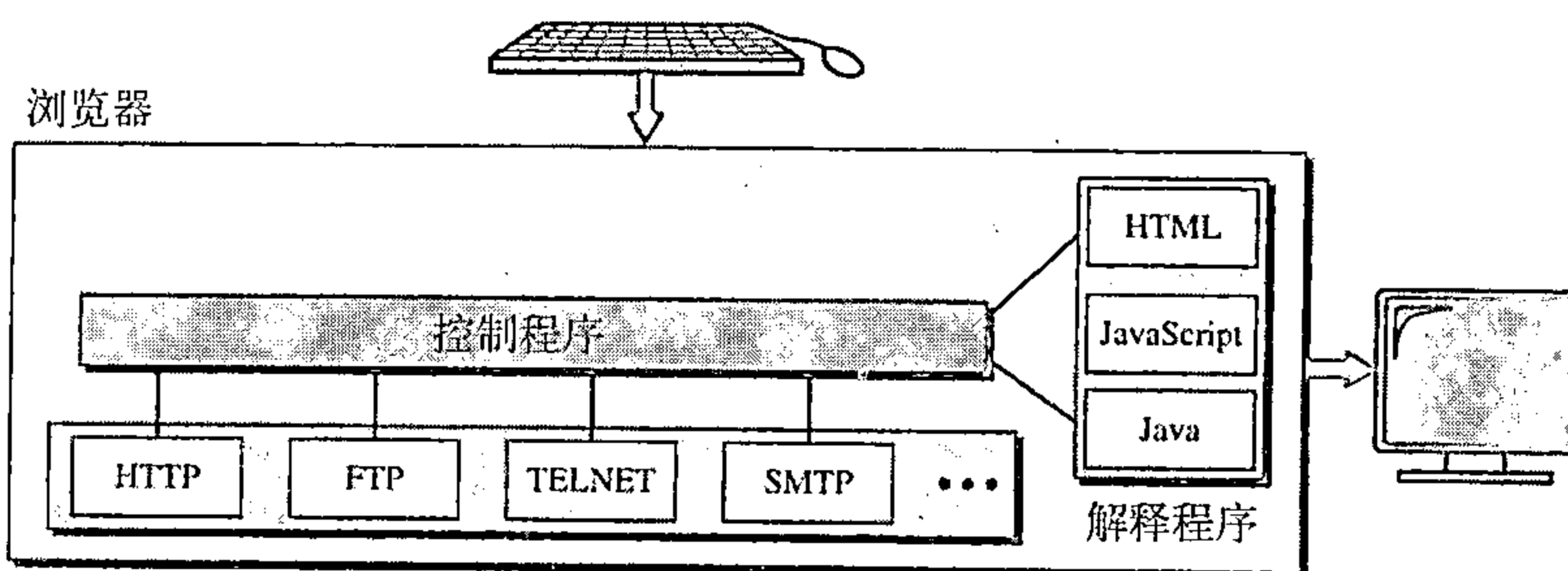


图27.2 浏览器

27.1.3 统一资源定位符

要访问Web页面的客户必须有一个地址。为了更方便地访问分布在世界各地的文档，HTTP使用了定位符。统一资源定位符（uniform resource locator, URL）是说明因特网上信息类型的一种标准。URL格式包括4个部分：协议、主机、端口和路径。（见图27.3所示）。



图27.3 URL

协议是客户/服务器程序用来检索文档的程序。许多不同的协议都能够检索文档；其中有FTP和HTTP。现在最常用的是HTTP。

主机（host）是信息所在的计算机，尽管计算机的名字可能是一个别名。Web页面通常存储在计算机中，计算机通常会赋予一个以字符“www”开头的别名。这并不是强制性的，因为主机可能是驻留Web页面的计算机的任何名字。

URL可以选择是否包含服务器的端口号。如果包含了端口，则将它插入在主机和路径之间，用冒号将它与主机分开。

路径（path）是信息所存储的文件的名称。注意：路径可以包含斜杠。在UNIX操作系统中，斜杠用来分隔目录和子目录及文件。

27.1.4 Cookies

万维网最初被设计成一个无国界的实体。客户端发送请求；服务器响应。它们之间的关系就结束了。最初设计的WWW，公开地检索可用的文档，完全符合这个目标。现在Web还有其他的功能，如下所示：

1. 有些网站只允许注册客户才能访问；
2. 网站作为电子商店，允许客户在商店内浏览，选择需要的商品，把它们放入电子购物车内，最后使用信用卡付费；
3. 有些网站是门户网站：用户可以选择他想看的Web页面；
4. 有些网站仅作为广告。

为了实现这些目的，cookie机制就应运而生。我们在第23章中的传输层讨论了cookies的使用，现在我们讨论它在Web页面中的使用。

Cookies的创建和存储

Cookies的创建和存储与实现有关，然而，它的原理是相同的。

1. 当服务器从客户端接收到请求后，它将有关客户端的信息存储在文件或字符串中。这些信息可能包含客户端的域名、cookie的内容（服务器收集到的关于客户端的信息，如主机名、注册号等）、时间戳、以及与实现有关的其他信息；

2. 服务器在响应中包含了它发送给客户机的cookie;
3. 当客户端接收到响应后, 浏览器在cookies目录中存储cookie, 并根据域服务器名进行分类。

Cookies的使用

当客户向服务器发送请求时, 浏览器在cookie目录中查询是否有从那个服务器发送过来的cookie。如果有, 则在请求中包含这个cookie。当服务器接收到这个请求后, 它就知道了这是一个老的客户, 而不是新的。注意: cookie的内容从来不让浏览器读取或者透露给用户, cookie只由服务器创建并收回。现在让我们分析如何使用cookie来实现上面提到的4个功能。

1. 当客户端第一次注册时, 网站就向客户端发送一个cookie; 网站通过这种方式限制注册用户的访问。只有那些能够发送正确cookie的客户才能被允许今后重复的访问。

2. 网上电子商店(电子商务)可以为客户端的购物者使用cookie。当客户端选择商品, 并放入到购物车中后; 包含了这些商品信息的(如它的数量、单价) cookie就被发送到浏览器。如果客户端选择第二个商品, cookie就被新的选择信息更新, 依次类推。当客户端结束购物并准备付账离开时, 就检索最终的cookie, 然后计算出总的费用。

3. Web门户以同样的方式使用cookie。当用户选择她最喜爱的页面时, 就生成一个cookie并发送到浏览器。当这个网站再次被访问时, 这个cookie就发送给服务器说明这个客户端要查找什么页面。

4. cookie也用来作为广告代理。广告代理能够将大字标题广告放置在用户经常访问的网站的主页面上。广告代理仅提供指出大字标题广告地址的URL, 而不是大字标题广告本身。当用户访问网站主页并点击广告公司的图标时, 一个请求就发送给了广告代理; 广告代理就发送一个大字标题广告, 如GIF文件, 同时也包含了一个含有用户ID的cookie。这个大字标题广告将来的任何使用都会加入到一个分析用户Web行为的数据库中。广告代理已经收集了用户的爱好, 并能够将这些信息卖给其他的组织。这种cookie的使用方法引起很多争议, 但愿今后能引入一些新的法规来保护用户的隐私信息。

27.2 Web文档

WWW中的文档可以分为三大类: 静态文档、动态文档和活动文档。基于文档内容的时间确定分类。

27.2.1 静态文档

静态文档 (static document) 是固定内容的文档, 它由服务器创建和并存储在服务器中。客户端只能获取文档的一份副本。换句话说, 文件的内容在创建时就已经确定了, 而不是在它被使用时。当然, 服务器中的内容也是可以改变的, 但用户不能改变它。当客户端访问静态文档时, 服务器会发送一份文档的副本, 用户可随后使用浏览器程序来显示这个文档 (见图27.4所示)。

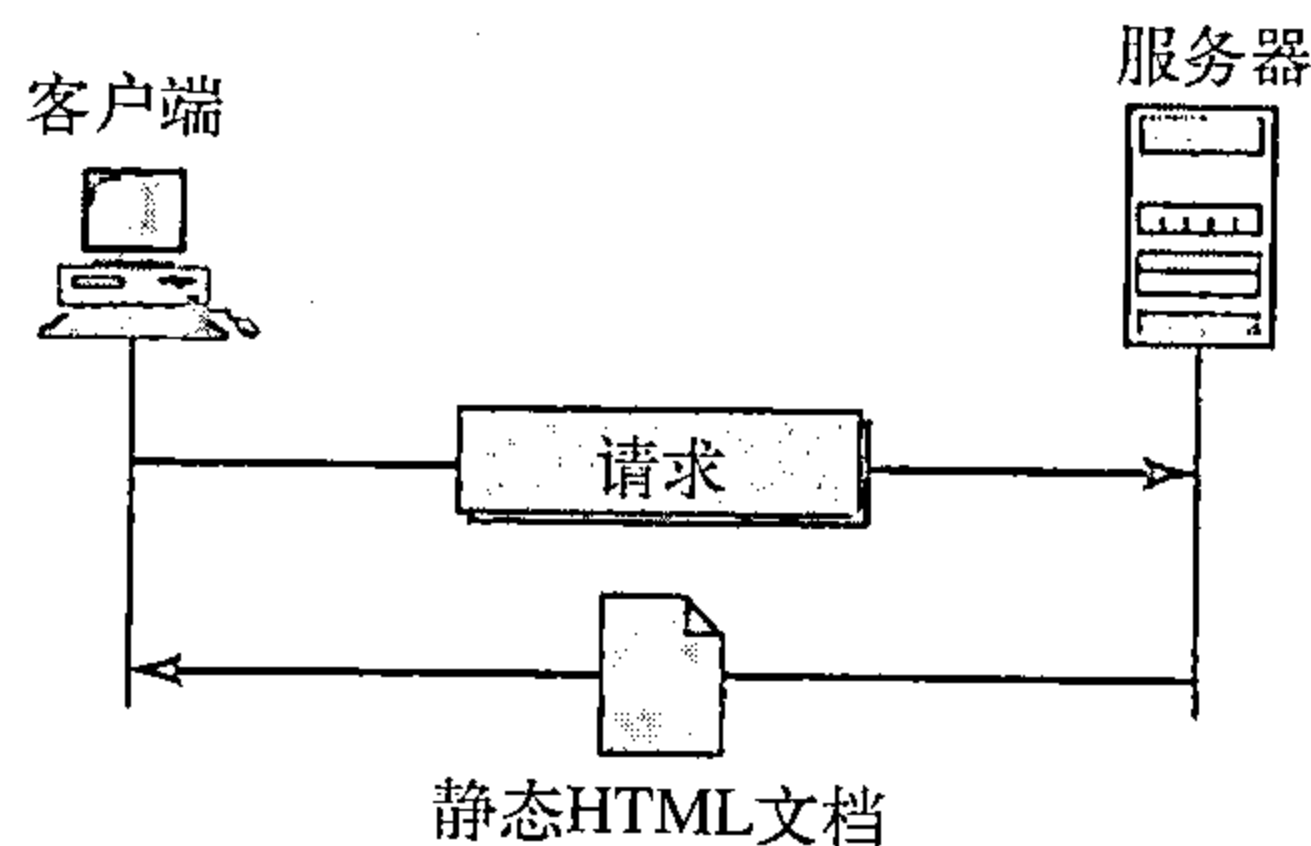


图27.4 静态文档

超文本标记语言

超文本标记语言 (Hypertext Markup Language, HTML) 是用于创建Web页面的语言。术语标记语言, 来自于书籍出版业。一本书在排版和印刷之前, 文字编辑阅读手稿, 在上面做

一些标记。这些标记告诉具体的工作人员如何处理文本。例如，文字编辑想要一行中的某一部分加粗字体，则会在这一部分上划一条波浪线。同样道理，Web页面的数据也需要进行格式化，以便浏览器进行解释。

我们用一个例子来阐述这一想法。要使一个文本的部分文本使用HTML显示为粗体，必须在文本的开始和结尾处放置粗体标签（标记），如图27.5所示。



图27.5 粗体标签

两个标签和是浏览器指令。当浏览器检查到这两个标记时，就知道两者之间的文本需要加粗显示（见图27.6所示）。

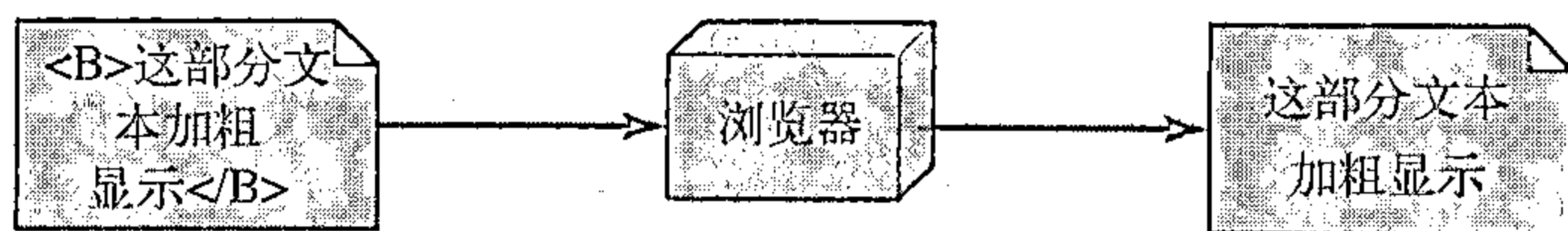


图27.6 粗体标签的效果

如HTML这样的标记语言允许我们在文件中嵌入格式化的指令，这些指令存储在文本中。在这种方式下，任何浏览器能够读取这些指令，并且根据特定的工作站对文本进行格式化处理。读者可能要问，为什么不使用字处理软件的格式功能来创建和存储格式化的文本？答案是不同的字处理软件使用不同的技术和过程。例如，假设一个用户使用Macintosh计算机创建了格式化的文本并存储在Web页面中。另一个使用IBM计算机的用户则不能接收这一Web页，因为两台计算机使用不同的格式化处理过程。

HTML在正文和格式化指令中都只使用ASCII字符。通过这种方式，每一台计算机能够按ASCII文档的形式接收整个文档。正文是数据，而格式化指令由浏览器用来对数据进行格式化处理。

Web页面由两部分组成：头部和主体。头部是Web页面的第一部分，它包含页面标题和浏览器要用到的其他参数。页面的真正内容在主体内，包含了正文和标签。然而，正文是页面中包含的真正信息，而标签定义了文档的外观。每个HTML标签是一个名字，后面跟着可选的属性列表，所有的内容都嵌入在小于符号和大于符号之间（<和>）。

如果有属性的话，则属性后面跟着一个等于符号及属性的值。一些标签可以单独使用，有些必须成对使用。成对使用的标签称为起始标签和结束标签。起始标签由标签名开始，可以有属性和值；结束标签不可以具有属性和值，但必须在标签名前加上一个斜杠。浏览器根据文本中嵌入的标签对文本的结构做出判断。图27.7说明了标签的格式。

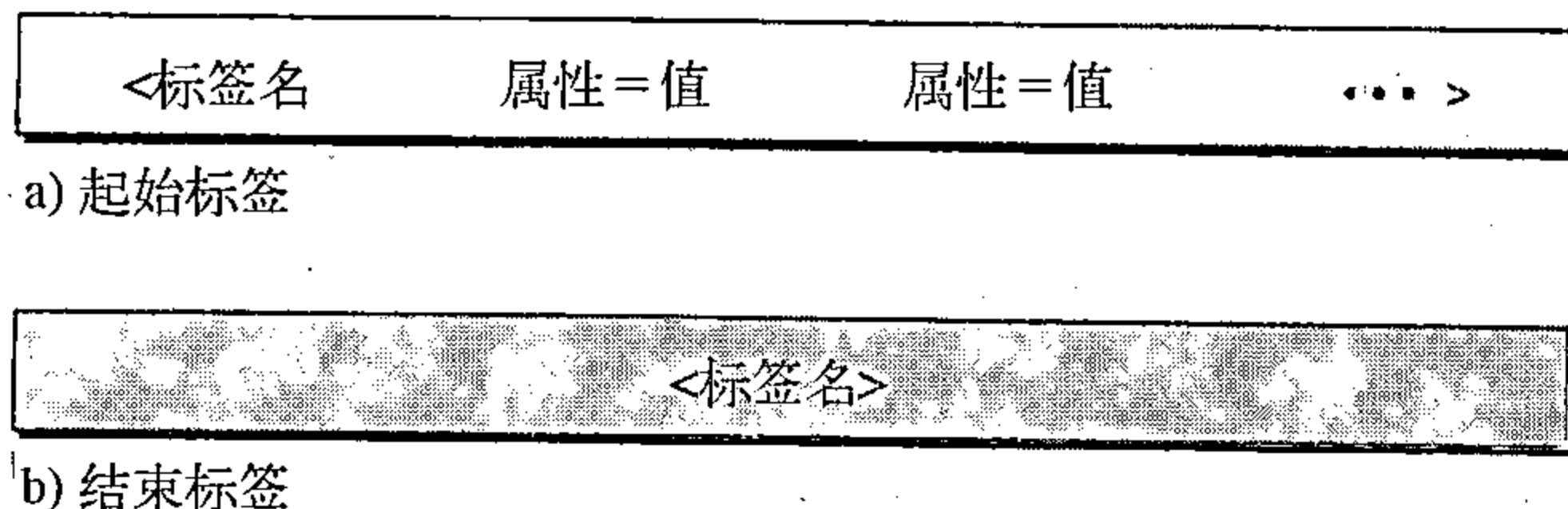


图27.7 起始标签和结束标签

通常使用的标签类型是文本格式标签，例如和，这使文本为粗体；<I>和</I>，这使文本为斜体；<U>和</U>，这使文本加下划线。

另外一个感兴趣的标签类型是图像标签。例如数字化照片或图像这样的非文本信息，这些并不是HTML文档的物理组成部分。但我们可以用图像标签来指明照片和图像文件。图像

标签定义了要检索图像的地址 (URL), 并且也指明了在找到图像之后如何插入到文本中。我们可以从许多属性中选择, 最常用的是SRC (源), 它定义源 (地址); 以及ALIGN, 它定义了图像的对齐。属性SRC是必须有的。大多数的浏览器接受GIF或者JPEG格式的图像。例如, 下面的标签可以在目录/bin/images里找到存储为image1.gif的图像:

```
<IMG SRC="/bin/images/image1.gif" ALIGN=MIDDLE>
```

第三类感兴趣的类型是超链接标签, 它将文档链接起来。通过一个称为锚的机制, 任何一个项目 (文字、短句、段落、或者图像) 都可以指向另一个文档。锚用标签<A...>和来定义。而锚定的项目通过URL指向另一个文档。当文档显示时, 锚定的项目出现下划线、闪烁、或者用粗体表示。用户可以点击锚定项目来访问另一个文档, 这个文档不一定与原先的文档存储在同一个服务器中。参照短句嵌入在起始标签和结束标签之间。起始标签可以有多个属性, 但其中的一个必须是HREF (超链接参考), 它定义了链接文档的地址 (URL)。例如, 对一本书的作者的链接可以是:

```
<A HREF="http://www.deanza.edu/forouzan"> Author </A>
```

在文本中显示的是单词“Author”, 但用户可以通过点击它访问该作者的Web主页。

27.2.2 动态文档

动态文档 (dynamic document) 是在有浏览器请求该文档时, 才由Web服务器创建的。当请求到达时, Web服务器运行一个应用程序或者脚本来创建动态文档。服务器返回程序或者脚本的输出作为对浏览器文档请求的响应。因为一个新的文档是根据每一个请求而创建的, 因此动态文档的内容是根据请求的不同而发生变化。动态文档非常简单的例子是获取服务器的时间和日期。时间和日期是一种每时每刻都在变化的动态信息。客户机可以请求服务器运行一个程序, 如UNIX中的date程序, 并将程序的结果发送给客户机。

公共网关接口 (CGI)

公共网关接口 (Common Gateway Interface, CGI) 是创建和处理动态文档的一种技术。CGI是一组标准, 它定义了如何编写动态文档, 如何将数据传递给应用程序, 以及如何使用输出结果。

CGI不是一种新的语言, 它允许程序员使用多种语言中任何一种, 如C、C++、Bourne 外壳、Korn 外壳、C外壳、TCL或者Perl。CGI只是定义了一组程序员应该遵循的规则和术语。

在CGI中, 术语“公共”是指这个标准定义的规则集对任何语言或者平台都是通用的。术语“网关”在这里表示CGI程序可以用来访问其他资源, 如数据库和图形包等。术语“接口”在这里是指有一组预先定义的术语、变量、调用等等都可以在任何CGI程序中使用。最简单的CGI程序的形式是用支持CGI的一种语言书写的代码。任何程序员, 只要有程序设计思想, 并且知道上面提到的任何一种语言的语法, 就能够编写出简单的CGI程序。图27.8说明了使用CGI技术创建动态程序的步骤。

输入 (Input) 在传统的编程设计中, 当程序执行时, 可以将参数传递给程序。参数传递机制允许编程设计者编写出在不同情况下能够使用的通用程序。例如, 一个通用的复制程序可以编写为将任何文件复制到另一个文件中。用户可以使用这个程序将名为x的文件复制为另一个名为y的文件时, 需要传递x和y作为参数。

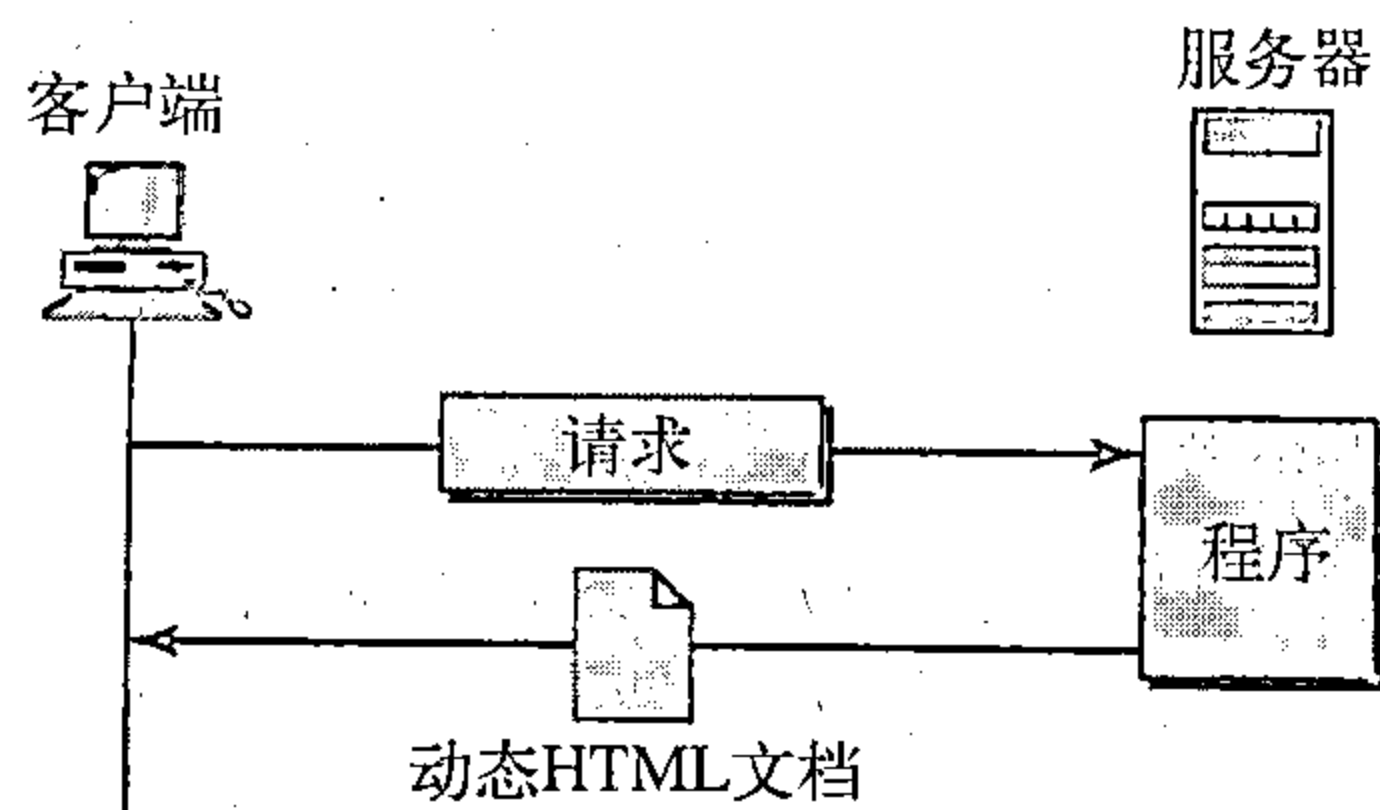


图27.8 使用CGI技术的动态文档

从浏览器到服务器的输入是通过使用一个表单 (form) 发送的。如果在表单中的信息比较短 (如一个字), 那么它可以通过一个问号附加在URL后面。例如, 下面的URL携带了表单的信息 (一个数值23):

`http://www.deanza/cgi-bin/prog.pl?23`

当服务器接收到这个URL时, 它利用URL问号前面的部分访问将要执行的程序, 然后解析问号后面的部分 (23) 作为客户端发送的输入信息。服务器将这个字符串存储一个变量中。当CGI程序执行时, 它可以读取这个值。

如果从浏览器来的输入信息太长而无法放入查询字符串中时, 浏览器可以请求服务器发送一个表单, 浏览器能够用输入数据填入这个表单中, 并发送给服务器。表单中的信息可以作为CGI程序的输入。

输出 (output) CGI总体的思想是在服务器站点执行CGI程序, 并把输出结果发送给客户 (浏览器)。输出的结果通常是普通文本或者是带有HTML结构的文本; 然而, 输出结果也可以是其他形式的对象。它可以是图像或者二进制数据、状态码、通知浏览器用来对结果高速缓存的指令、或者通知服务器用来发送现有的文档而不是真正输出的指令。

为了让客户端知道发送文档的类型, CGI程序必须创建头部。实际上, CGI程序的输出总是包括两个部分: 头部和主体。头部与主体之间用空行隔开。这意味着任何一个CGI程序首先创建头部, 然后是一个空行, 最后是主体。尽管在浏览器屏幕上不显示头部和空行, 但浏览器要使用头部对主体进行解释。

动态文档的脚本技术

CGI技术的问题是低效率, 如果要创建的动态文档部分的内容是固定的, 并且不随请求的不同而发生变化, 这时候低效率就能体现出来。例如, 假设我们需要检索一个特定汽车品牌的备用零件列表, 它们的有效性及价格。尽管有效性和价格经常会发生变化, 但是名字、描述以及零件的图片是固定不变的。如果我们使用CGI, 每次一个请求生成后, 程序必须创建一个完整的文档。创建一个包含文档固定部分的文件的解决方法是使用HTML, 并嵌入脚本和源码, 服务器运行的是提供可变化的有效性及价格部分。图27.9说明了这个概念。

一些技术已经被引入到使用脚本来创建动态文档。其中最常见的是: 超文本预处理器 (Hypertext Preprocessor, PHP), 它使用Perl语言; Java服务器网页 (Java Server Pages, JSP) 技术, 它使用Java语言作为脚本;

活动服务器网页 (Active Server Pages, ASP) 技术, 这是微软的产品, 它使用VB语言作为脚本; 以及ColdFusion, 它在HTML文档中嵌入了SQL数据库查询。

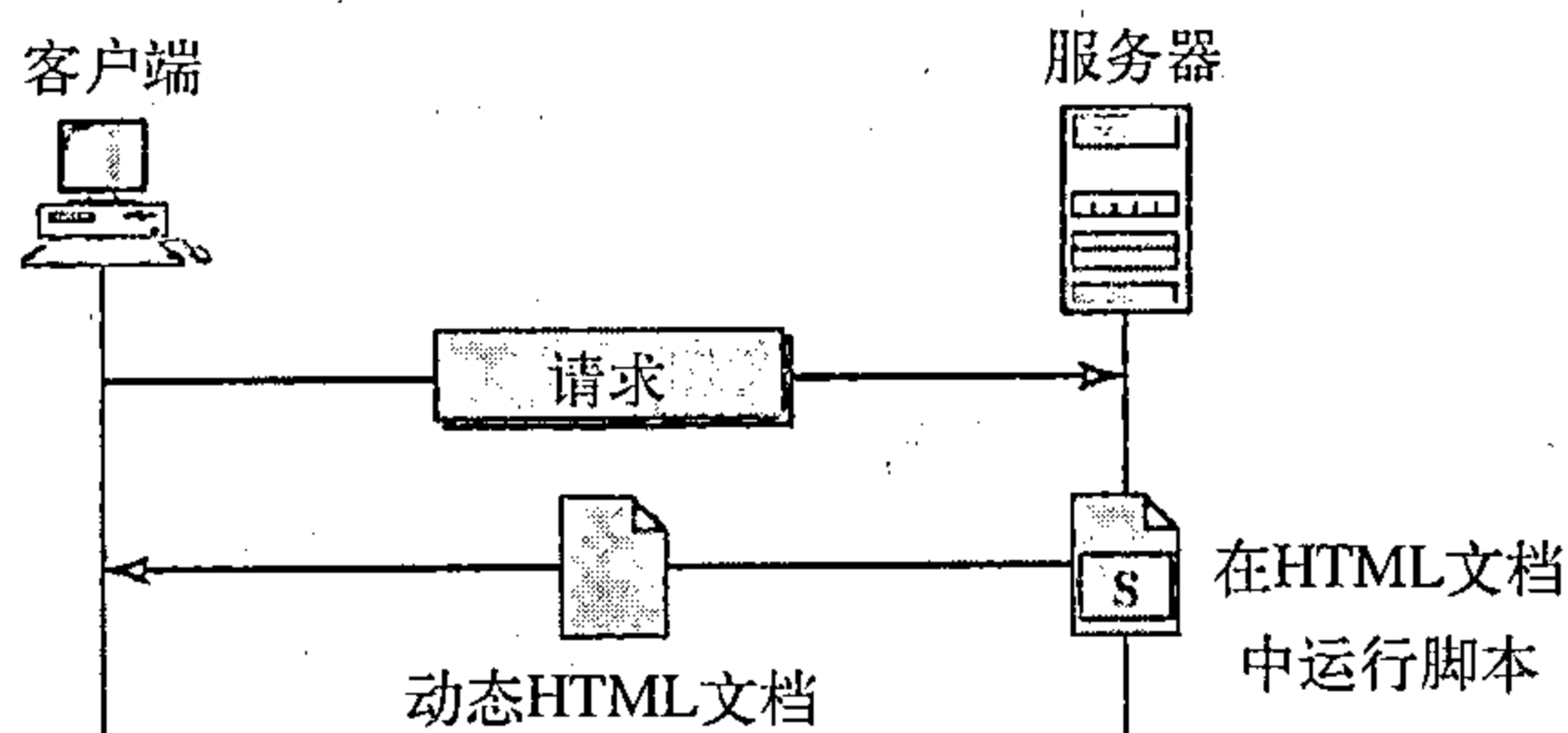


图27.9 使用服务器端脚本的动态文档

有时, 动态文档可以称为服务器端的动态文档。

27.2.3 活动文档

对于许多应用来说, 我们需要程序或者脚本在客户端运行。这些称为活动文档 (active document)。例如, 假设我们要运行在屏幕上创建动画图像与用户进行交互的程序。很明显, 这个程序需要在客户端运行, 也就是在动画和交互发生的地方运行。当浏览器请求活动文档时, 服务器发送一份文档或者一个脚本的副本。然后文档就在客户端 (浏览器) 运行。

Java小应用程序

使用Java小应用程序 (applets) 是创建活动文档的一种方式。Java是一种高级编程语言、一个实时运行环境和一个类库的组合。程序员可以使用Java编写活动文档 (一个小应用程序)，并在浏览器中运行。它也可以是不使用浏览器的独立运行的应用程序。

applet是在服务器端用Java编写的应用程序，它是已经编译的、并且是可运行的程序。文档是字节码 (二进制) 格式的，客户端处理器 (浏览器) 创建了这个applet的实例并开始运行。用浏览器运行Java applet有两种方式。第一种方式：浏览器可以直接地在URL中请求java applet程序，并以二进制格式接收applet；第二种方式：浏览器可以找到并运行以applet地址嵌入为标签的HTML文件。图27.10说明了在第一种方式下，如何使用java applet；第二种方式是类似的，但需要两个事务。

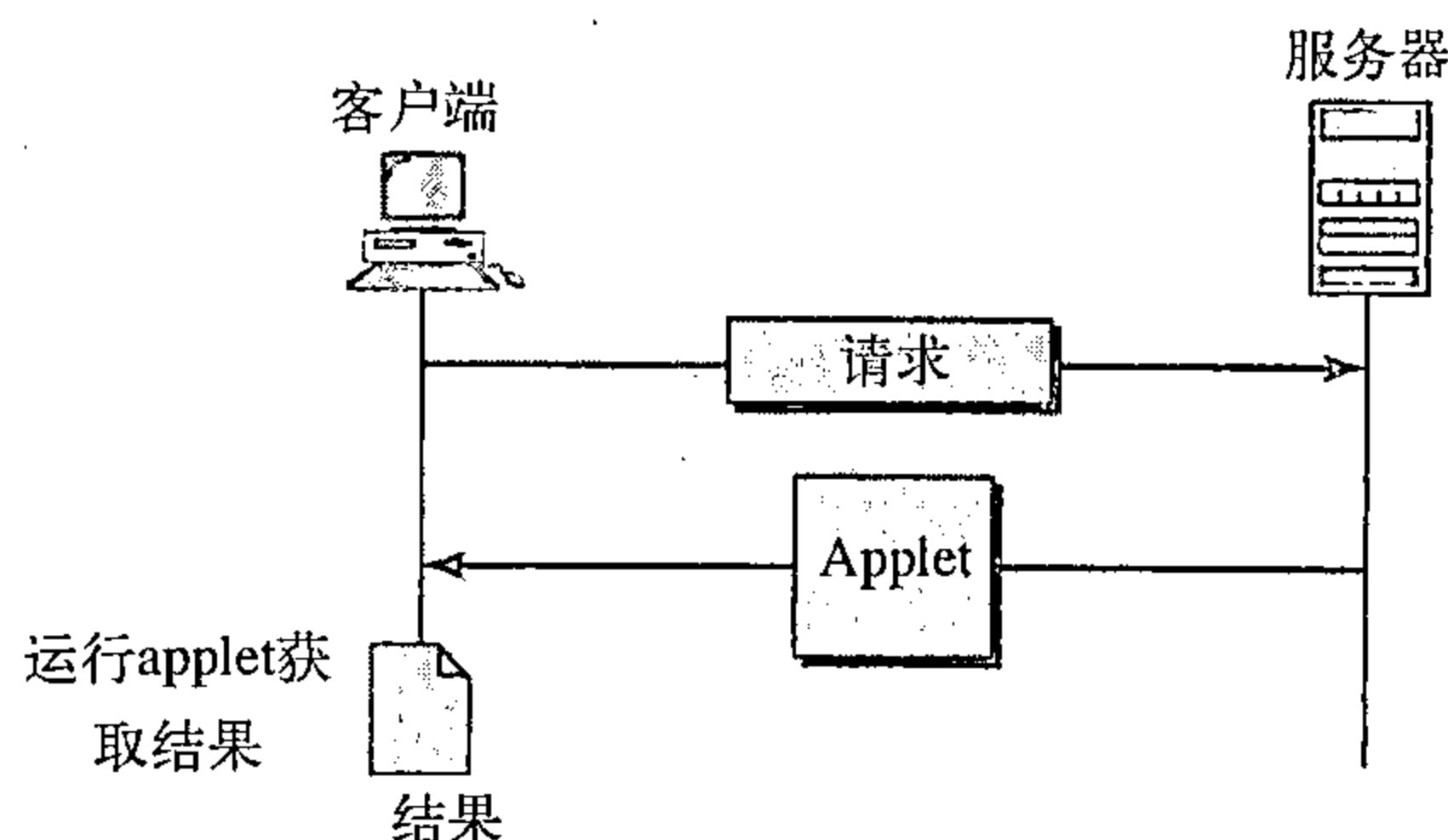


图27.10 用Java applet活动文档

JavaScript

在动态文档中的脚本思想也可以用在活动文档中。如果文档中的活动部分很少，那么它可以用脚本语言编写，然后在客户端运行并同时解释。脚本是源码形式 (文本)，而不是二进制格式。在这种方式下使用的脚本技术一般来说就是JavaScript。JavaScript (Java脚本) 是与Java完全不同的语言，它是为这个目的而发展起来的高级脚本语言。图27.11说明了如何用JavaScript创建活动文档。

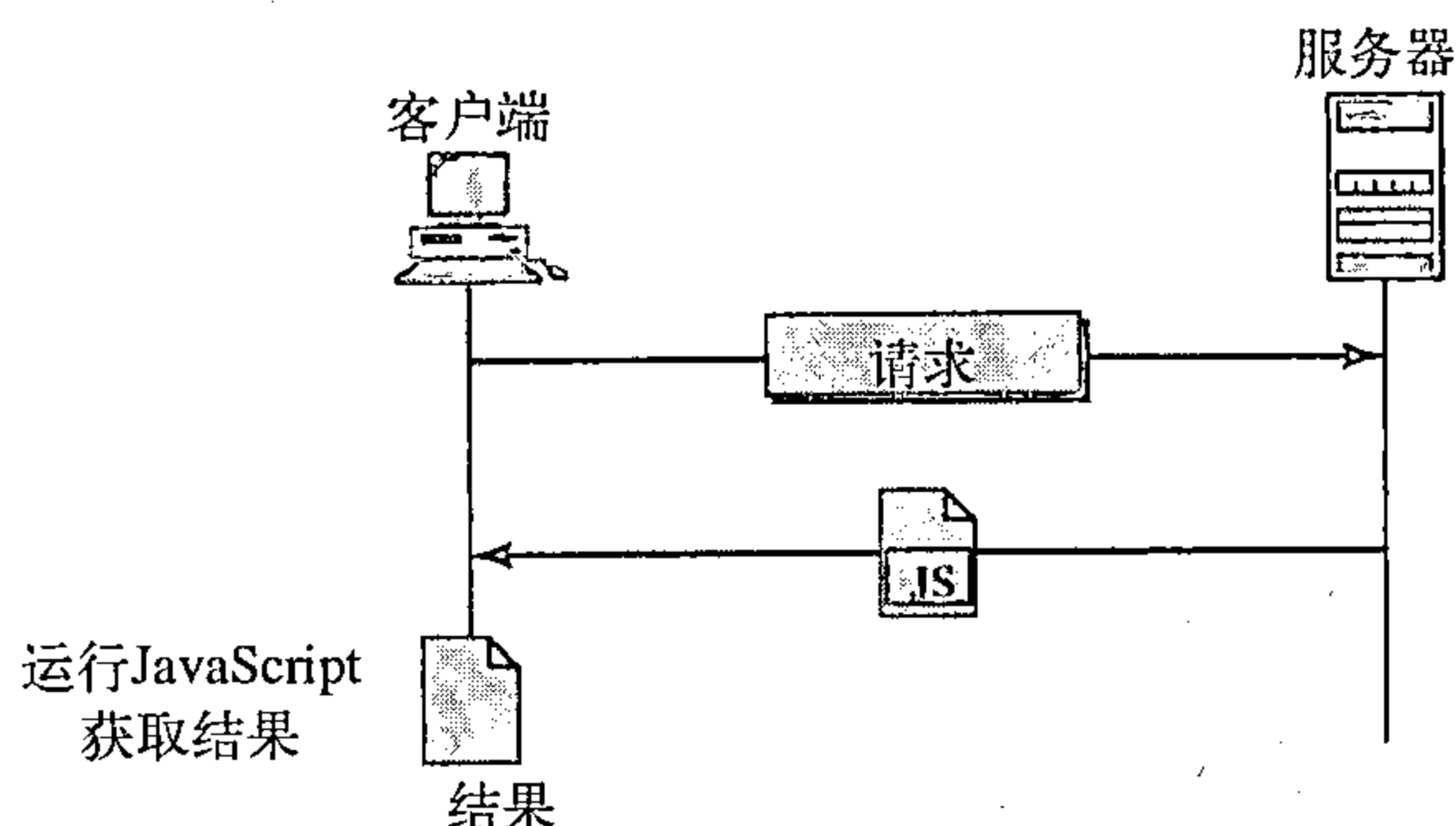


图27.11 使用客户端脚本的活动文档

活动文档有时候可以认为是客户端的动态文档。

27.3 超文本传输协议 (HTTP)

超文本传输协议 (Hypertext Transfer Protocol, HTTP) 主要用于万维网上存取数据的协议，HTTP在功能像是FTP和SMTP的组合。它与FTP相似，是因为它能够传输文件并使用TCP服务。但是，它比FTP更简单，因为它只使用了一个TCP连接。它没有单独的控制连接；在客户机和服务器之间只传输数据。

HTTP也与SMTP相似，是因为在客户端和服务器之间传输的数据与SMTP报文很相似。此外，报文格式由类似MIME的头部控制。HTTP与SMTP不同的是，HTTP报文不是供人阅读的，它们是由HTTP服务器和HTTP客户端 (浏览器) 读取并解释。SMTP报文是存储转发的，而HTTP报文需要即时发送。从客户端发送到服务器的命令嵌入在请求报文中。而请求文件的内容或其他信息嵌入在响应报文中。

HTTP在熟知端口80上使用TCP服务。

27.3.1 HTTP事务

图27.12说明了客户端和服务端之间的HTTP事务。尽管HTTP使用TCP服务，但是HTTP本身

是一种无状态协议。客户端通过发送请求报文初始化这个事务，服务器通过发送响应进行回复。

报文

请求和响应报文的格式是相似的，图27.13对两者都做了说明。请求报文由一个请求行、一个头部构成，有时还可能包括主体。响应报文由状态行、头部构成，有时还可能包含主体。

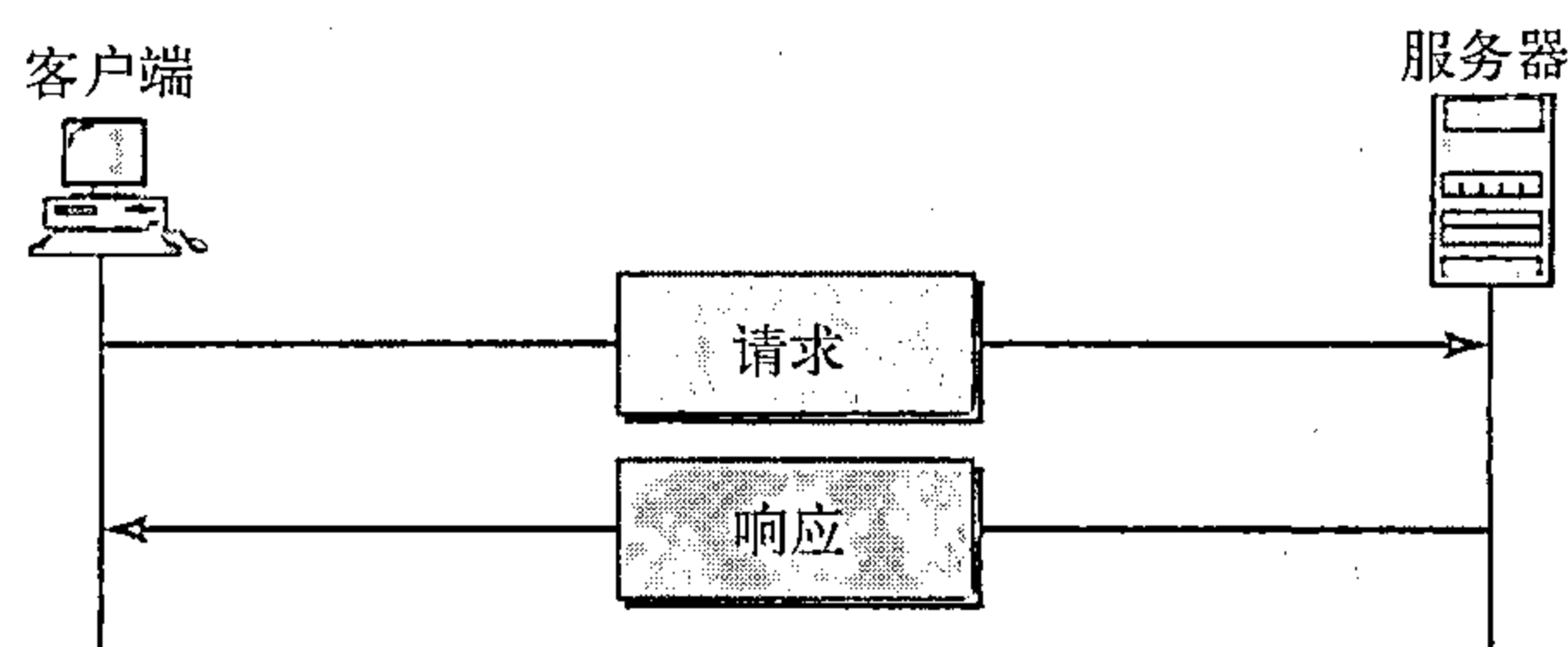


图27.12 HTTP事务

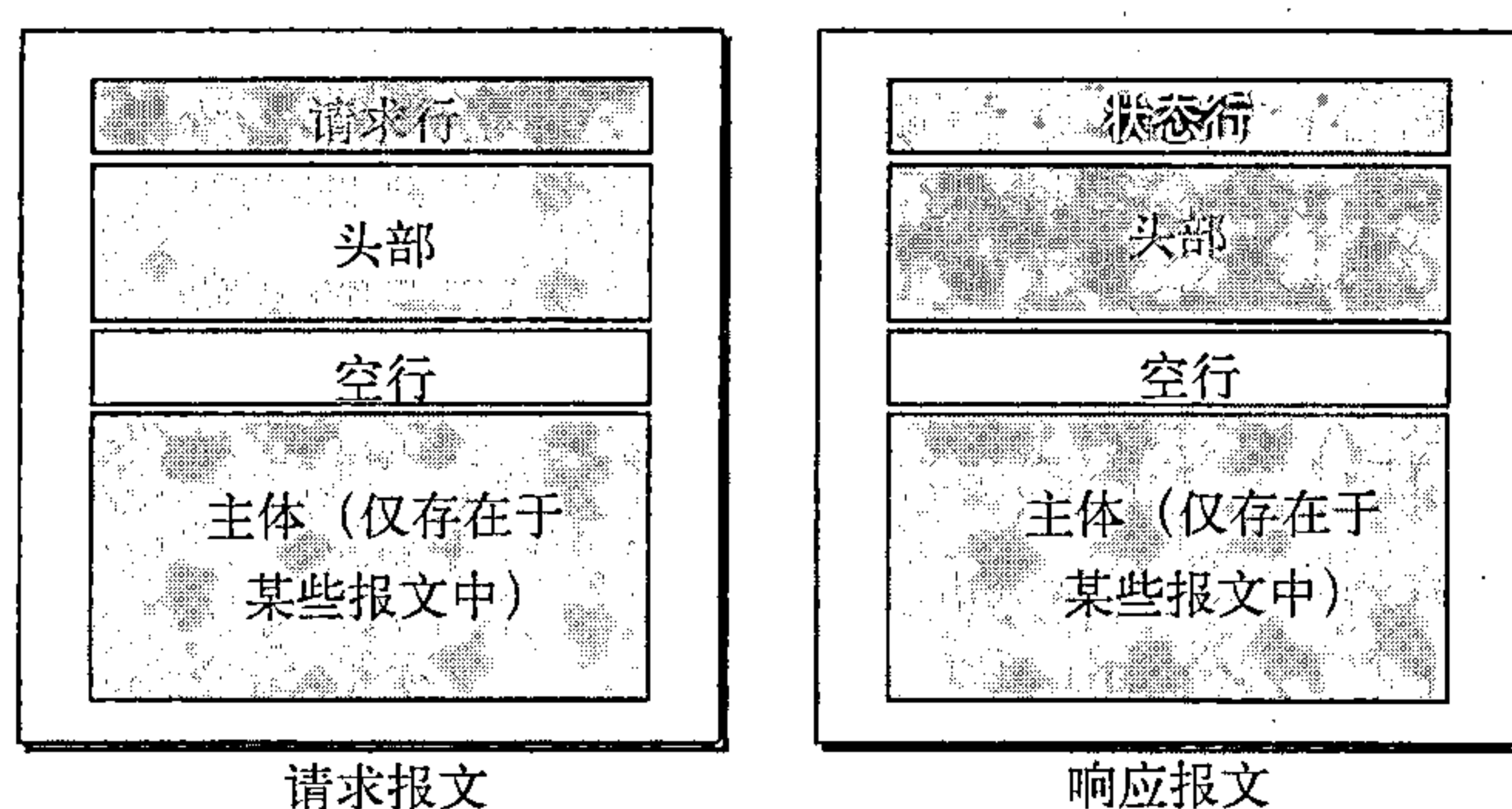


图27.13 请求与响应报文

请求和状态行 在请求报文中的第一行称为请求行（request line），在响应报文中的第一行称为状态行（status line）。两者有一个共同的字段，如图27.14所示。

- **请求类型**。这个字段在请求报文中使用。在HTTP 1.1版本中，定义了几种请求类型。请求类型（request type）分类成几种方法，如表27.1所示。

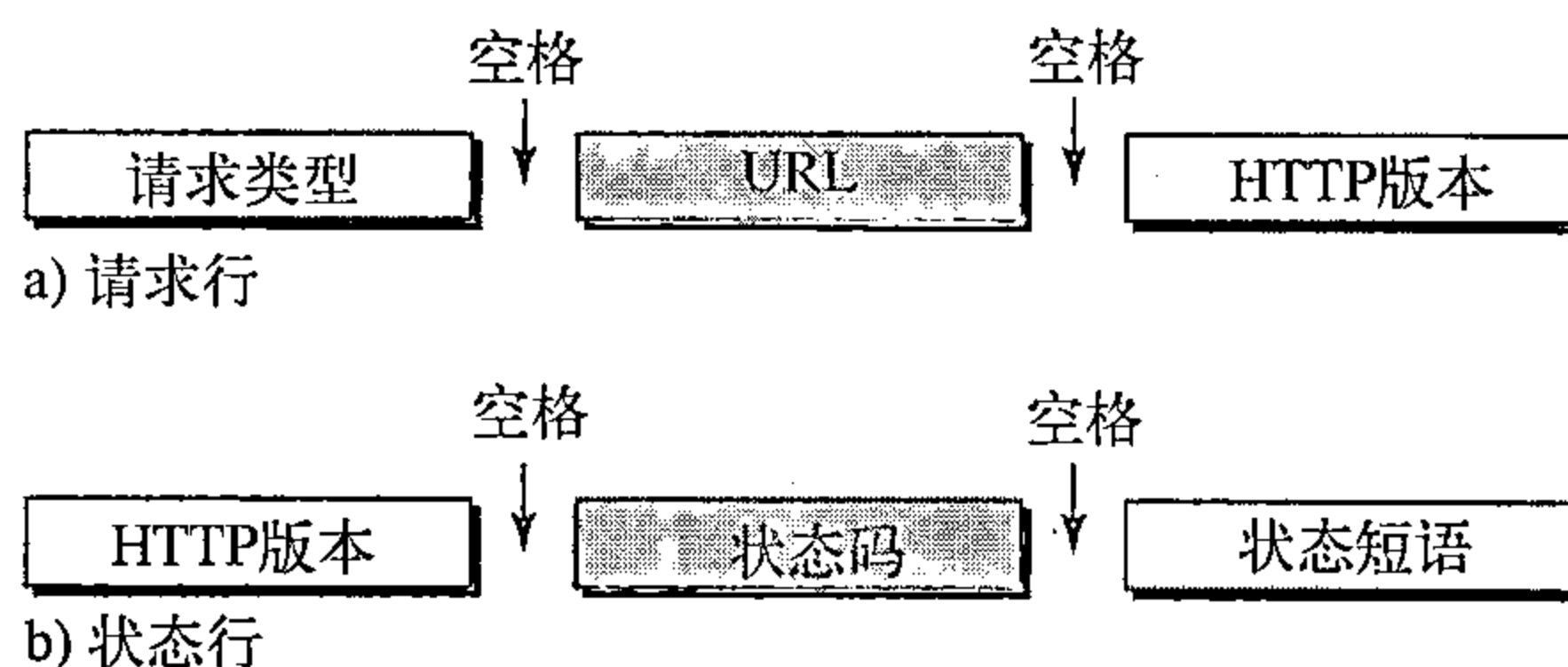


图27.14 请求和状态行

表27.1 方法

方 法	动 作	方 法	动 作
GET	向服务器请求文档	TRACE	回显输入的请求
HEAD	请求关于文档的信息，而不是文档本身	CONNECT	预留
POST	从客户端向服务器发送信息	OPTION	询问有关可用的选项
PUT	从服务器向客户端发送文档		

- **URL**。在本章的前面部分，我们已经讨论了统一资源定位符（URL）。
- **版本**。HTTP目前最常用的版本是1.1。
- **状态码**。这个字段在响应报文中使用。状态码字段（status field）与在FTP、SMTP协议中的字段相似，它由三个数字组成。在100系列的代码只代表一个报告；在200系列的代

码表示这是一个成功的请求；在300系列的代码表示把客户端重定向到另一个URL；在400系列的代码表示在客户端发生错误；在500系列的代码表示错误发生在服务器端。表27.2列出了常用的代码。

- 状态短语。这个字段在响应报文中使用。它用文本格式解释了状态码。表27.2也给出了状态短语。

表27.2 状态码

代 码	短 语	描 述
提供信息		
100	Continue	请求的初始部分已经收到，客户端可以继续它的请求
101	Switching	服务器同意客户的请求，切换到更新头部所定义的协议成功
200	OK	请求成功
201	Created	创建了一个新的URL
202	Accepted	请求已经接受，但它不能立即响应
204	No content	主体中没有内容
重定向		
301	Moved permanently	服务器已不再使用所请求的URL
302	Moved temporarily	请求的URL已经暂时移开
304	Not modified	文档还没有被修改
客户端错误		
400	Bad request	在请求中有语法错误
401	Unauthorized	请求缺乏合适的授权
403	Forbidden	服务被拒绝
404	Not found	没有找到文档
405	Method not allowed	URL不支持该方法
406	Not accept able	不接受这种格式的请求
服务器错误		
500	Internaluser error	服务器有错误，例如系统崩溃
501	Not implemented	请求的动作不能完成
503	Service unavailable	服务暂时不可用，但将来可以被再次请求

头部 (header) 头部用于客户端和服务端之间交换附加的信息。例如，客户端可以请求以某种特殊格式发送文件，或者服务器发送与文档有关的附加信息。头部由一行或者多行组成，每一行由一个头名字、一个冒号、一个空格和头值组成（见图27.15所示）。在本节的后面，我们将举一些例子说明头部的各行。头部的各行属于下列四类之一：**通用头部** (general header)、**请求头部** (request header)、**响应头部** (response header) 和 **实体头部** (entity header)。请求报文只能包含通用、请求和实体头部。响应报文只能包含通用、响应和实体头部。

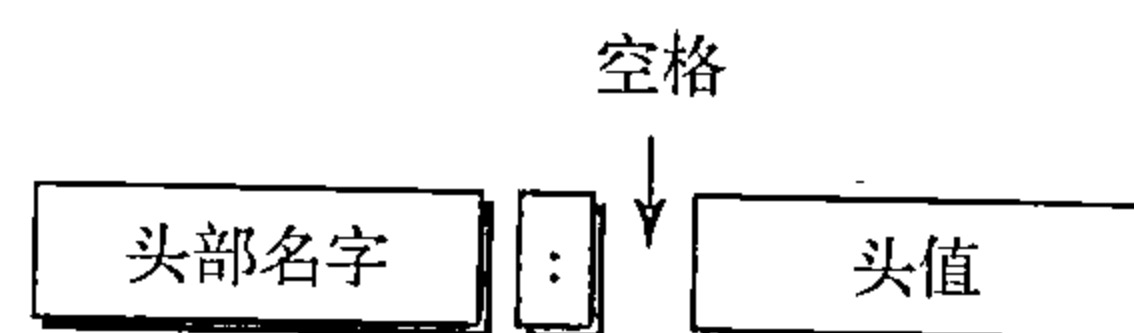


图27.15 头部格式

通用头部给出关于报文的通用信息，并可以在请求和响应报文中都存在。表27.3列出了一些通用头部及其说明。

表27.3 通用头部

头 部	说 明
Cache-control	定义了关于高速缓存的信息
Connection	说明连接是否应关闭
Date	给出现在的日期
MIME-version	说明MIME的使用版本
Upgrade	确定首选的通信协议

请求头部只能在请求报文中存在。它确定客户端配置和客户端首选的文档格式。表27.4列出了一些请求头部及其说明。

表27.4 请求头部

头 部	说 明
Accept	给出客户端能够接受的媒体格式
Accept-charset	给出客户端可以处理的字符集
Accept-encoding	给出客户端可以处理的编码方案
Accept-language	给出客户端可以接受的语言
Authorization	给出客户端有哪些许可
From	给出用户的电子邮件地址
Host	给出主机及服务器的端口号
If-modified-since	只在比指定的日期更新时，才发送文档
If-match	只有与给定的标记匹配时，才发送文档
If-non-match	只有与给定的标记不匹配时，才发送文档
If-range	只发送丢失的那部分文档
If-unmodified-since	如在指定的日期之后未改变，则发送文档
Referrer	指明被链接文档的URL
User-agent	标识客户端程序

响应头部只能出现在响应报文中。它确定了服务器的配置和关于请求的特定信息。表27.5列出了一些响应头部及其说明。

表27.5 响应头部

头 部	说 明
Accept-range	说明服务器是否接受客户端请求的范围
Age	给出文档的使用年限
Public	给出支持的方法列表
Retry-after	确定一个日期，在这个日期之后服务器是可用
Server	给出服务器名及版本号

实体头部提供了关于文档主体的有关信息。尽管多数情况下它只出现在响应报文中，但在有些请求报文中，如POST和PUT方法，也会包含一个主体且使用这种类型的头部。表27.6列出了一些实体头部及其说明。

表27.6 实体头部

头 部	说 明
Allow	列出URL可以使用的有效方法
Content-encoding	确定编码方案
Content-language	指定语言
Content-length	给出文档的长度
Content-range	指定文档的范围
Content-type	指定媒体类型
Etag	给出实体的标记
Expires	给出内容改变的日期和时间
Last-modified	给出上次改变的日期和时间
Location	指明新建或移动后文档的位置

主体 (body) 可以出现在请求和响应报文中。通常, 它包含要发送或接收到的文档。

例27.1

这个例子是检索文档。我们用GET方法检索路径为/usr/bin/image1的图像。请求行给出方法 (GET)、URL和HTTP版本 (1.1)。其头部有两行, 以表明客户端可以接受GIF和JPEG格式的图像。请求报文没有主体。响应报文包含状态行和四行的头部。这些头部行定义了日期、服务器、MIME版本和文档的长度。文档的主体位于头部之后 (图27.16所示)。

例27.2

这个例子是客户端要向服务器发送数据, 我们使用了POST方法。请求行说明了方法 (POST), URL和HTTP版本 (1.1)。其头部有4行, 请求主体中包含了输入信息。响应报文包含了状态行和四行的头部。被创建的文档是一个CGI文档, 它包含在响应报文的主体中 (如图27.17所示)。

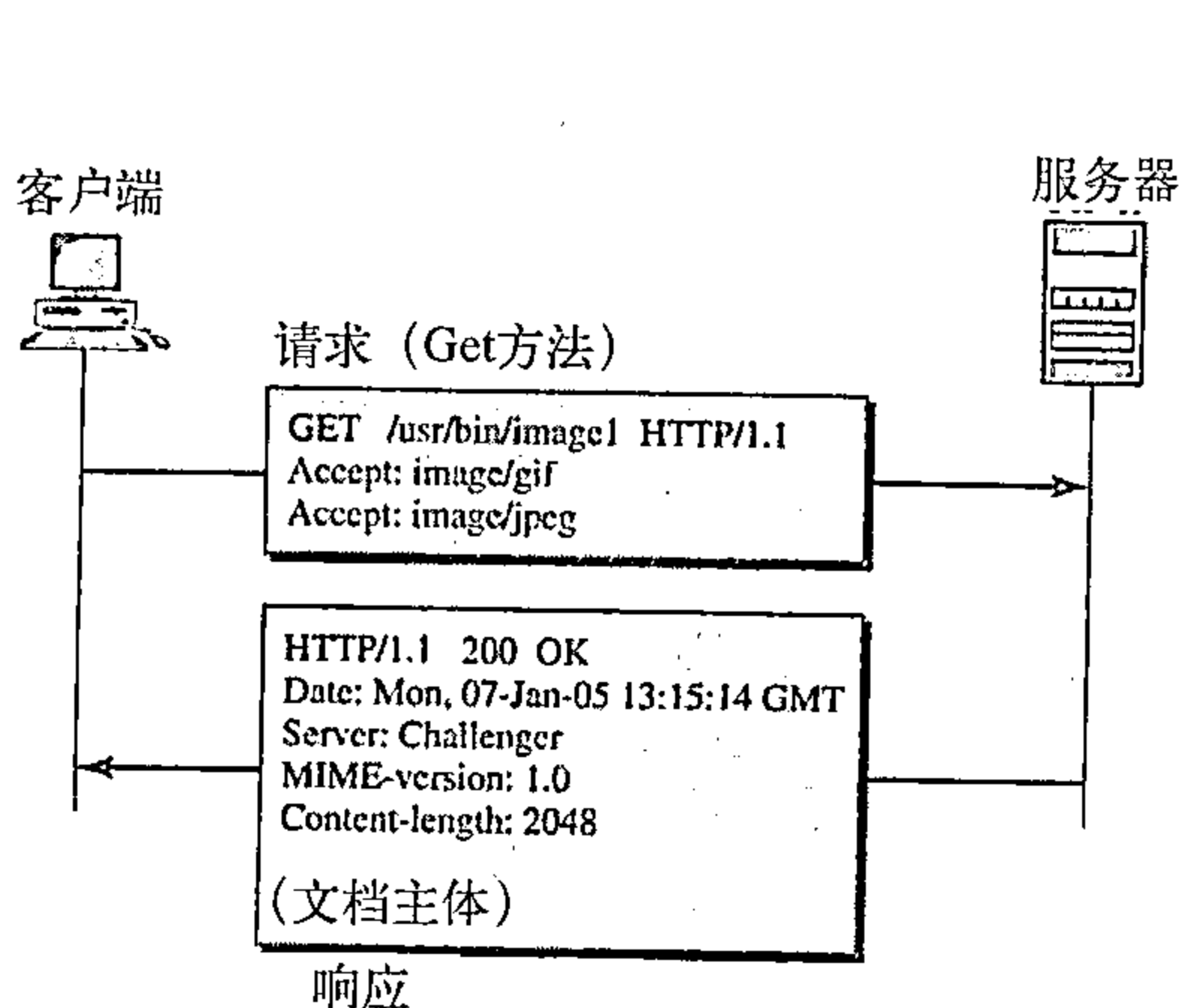


图27.16 例27.1

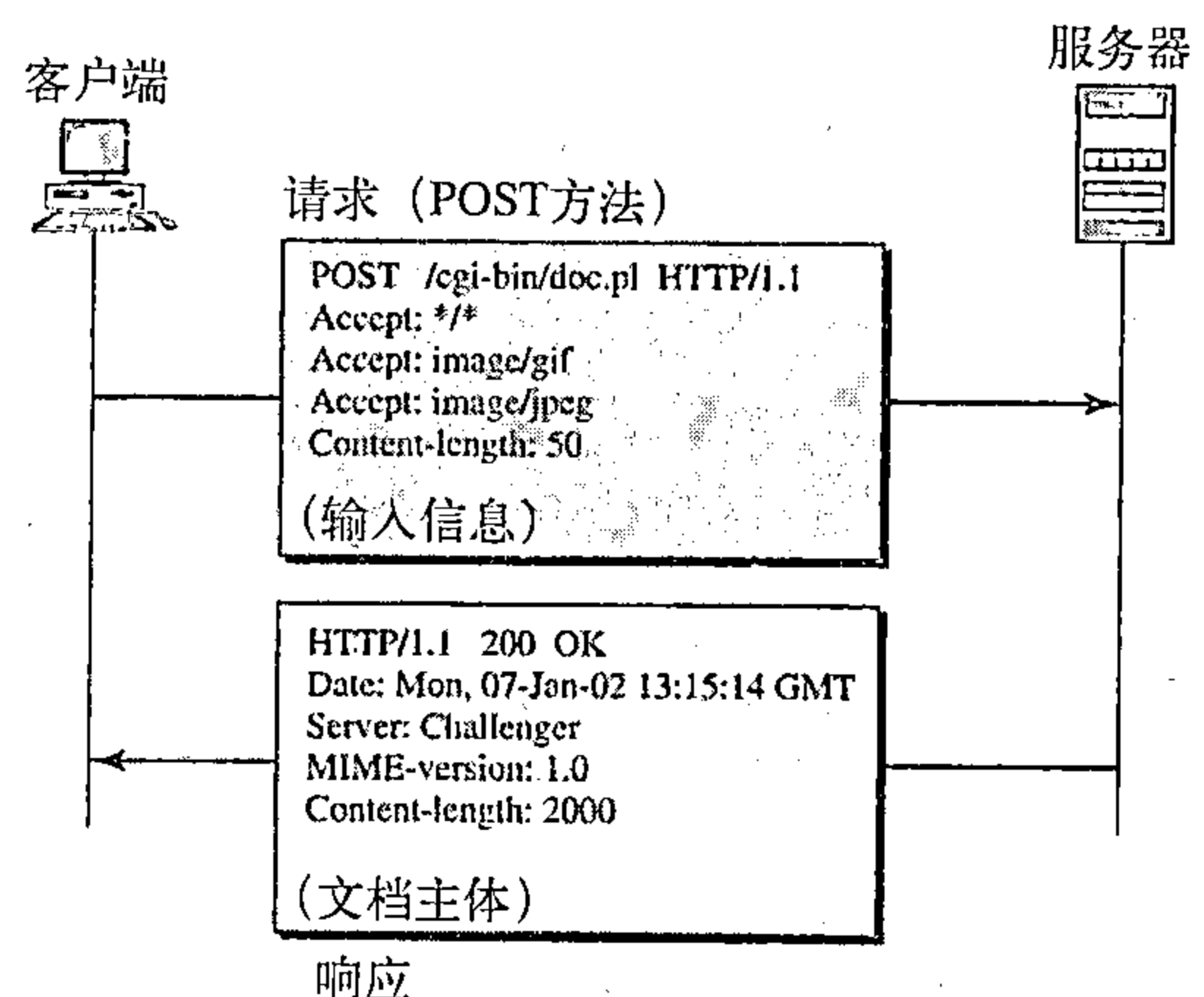


图27.17 例27.2

例27.3

HTTP使用ASCII字符, 客户端使用TELNET直接连接到服务器上, 登录端口号是80。接下来的三行说明了这个连接是成功的。

然后我们输入三行, 第一行给出请求行 (GET方法), 第二行给出头部 (定义了主机), 第三行是空行, 结束请求。

服务器响应是以状态行开始的7行信息。最后的空行作为服务器响应的结束。在空行之后是接收到的14 230行的文件 (在这里没有显示)。最后一行是客户端的输出。


```
$ telnet www.mhhe.com 80
Trying 198.45.24.104 ...
Connected to www.mhhe.com (198.45.24.104).
Escape character is '^]'.
GET /engcs/compsci/forouzan HTTP/1.1
From: forouzanbehrouz@fhda.edu

HTTP/1.1 200 OK
Date: Thu, 28 Oct 2004 16:27:46 GMT
Server: Apache/1.3.9 (Unix) ApacheJServ/1.1.2 PHP/4.1.2 PHP/3.0.18
MIME-version:1.0
Content-Type: text/html

Last-modified: Friday, 15-Oct-04 02:11:31 GMT
Content-length: 14230

Connection closed by foreign host.
```

27.3.2 持续与非持续连接

HTTP在版本1.1之前定义了非持续连接，而在版本1.1中默认的是持续连接。

非持续连接

在非持续连接（nonpersistent connection）中，每一次请求/响应都要建立TCP连接。下面是实现这一策略的步骤：

- 1、客户端建立TCP连接，并发送请求；
- 2、服务器发送响应，并关闭连接；
- 3、客户端读取数据，直到遇到文件结束标志，客户端随后关闭连接。

在这种策略中，对于不同文件中的N个不同的图片，连接必须建立和关闭N次。因为服务器需要N个不同的缓冲区，并且每次建立一个连接时，需要一个较慢的启动过程，所以非持续策略在服务器端增加了很大的开销。

持续连接

HTTP1.1定义了默认的持续连接（persistent connection）。在持续连接中，服务器在发送响应以后会保持连接处于开启状态，以等待更多的请求。如果客户请求关闭或者超时时，服务器会关闭连接。发送方通常在每次响应时会发送数据的长度。但是，在一些特殊情况下，发送方不能确定数据的长度。如果文档是动态创建的或者属于活动文件，就会发生这种情况。这种情况下，服务器会通知客户端，文件长度未知，发送数据结束后会关闭连接，这样客户端就能确定到达的数据是否已经结束。

HTTP1.1版本指定默认的连接是持续连接。

27.3.3 代理服务器

HTTP支持代理服务器（proxy server）。代理服务器是一台计算机，能够保存最近请求的响应的副本。HTTP客户端向代理服务器发送请求。代理服务器检查本机的高速缓存。如果高速缓存中不存在响应报文，代理服务器就向相应的服务器发送请求。返回的响应会发送到代理服务器中，并且进行存储，以用于其他客户端将来的请求。

代理服务器降低了原服务器的负载，减少了通信量并降低了延迟。但是，使用代理服务器，客户端必须配置为访问代理服务器而不是目标服务器。

推荐读物

为了深入讨论与本章有关的主题，推荐下列读物和网站。在本章末列出方括号[...]中的参

考资料。

读物

HTTP在[Ste96]的第13章和第14章、[PD03]的9.3节、[Com04]的第35章、[Tan03]的7.3节中讨论。

网站

下面的网站与本章中讨论的主题有关：

www.ietf.org/rfc.html关于RFC的信息

请求评论

下面是与WWW相关的RFC：

1614, 1630, 1737, 1738

下面是与HTTP相关的RFC：

2068, 2109

本章小结

- 万维网（WWW）是分布在世界各地互相链接在一起的信息仓库。
- 超文本是通过指针互相链接在一起的文档。
- 浏览器解释并显示文档。
- 浏览器由控制程序、客户端程序及解释程序组成。
- Web文档可以分为：静态文档、动态文档和活动文档。
- 静态文档是存储在服务器中的内容固定的文档，客户机不能改变服务器上的文档。
- 超文本标记语言（HTML）是用于创建静态Web页面的语言。
- 任何浏览器都能够读取嵌入在HTML文档中的格式化指令（标签）。
- 标签提供了文档的结构，定义了标题和头部，格式化文本，控制数据流，插入图、将不同的文档链接在一起，并且定义了可执行码。
- 只有在浏览器请求后，服务器才能创建动态Web文档。
- 公共网关接口（CGI）是创建、处理动态Web文档的标准。
- CGI程序和嵌入的CGI接口标签可以用多种语言编写，如C、C++、shell脚本语言或者Perl。
- 活动文档是客户端检索的程序的副本，在客户端运行。
- Java是一种高级程序设计语言、一种运行时环境及允许程序员编写活动文档的类库的组合。生成的活动文档可以在浏览器中运行。
- Java用来创建applet（小应用程序）。
- 超文本传输协议（HTTP）是万维网（WWW）存取数据的主要协议。
- HTTP使用TCP连接传输文件。
- HTTP报文在形式上与SMTP报文相似。
- HTTP请求行由请求类型、URL、以及HTTP版本号组成。
- 资源定位符（URL）是由方法、主机、可选的端口号以及在WWW中定位信息的路径名组成。
- HTTP请求类型或方法是由客户端向服务器发出的实际命令或请求。
- 状态行是由HTTP版本号、状态码和状态短句组成。

- HTTP状态码代表了通用的信息，这些信息与成功请求、重定向信息、错误信息相关。
- HTTP头部代表了在客户端与服务器之间的额外信息。
- HTTP头部由头部名字和头值组成。
- HTTP通用头部给出了关于请求和响应报文的通用信息。
- HTTP请求头部说明了客户端的配置和最佳的文档格式。
- HTTP响应头部说明了服务器的配置和关于请求的特殊信息。
- HTTP实体头部提供了关于文档主体的信息。
- HTTP1.1版本定义了持续连接。
- 代理服务器保存了最近请求的响应副本。

习题

复习题

1. HTTP与WWW之间有什么关系？
2. HTTP与SMTP的相似之处是什么？
3. HTTP与FTP的相似之处是什么？
4. 什么是URL？它由哪些部分组成？
5. 什么是代理服务器？它与HTTP有什么关系？
6. 说出浏览器共有的三个组成部分的名字。
7. Web文档有哪三种类型？
8. HTML的含义是什么？它的功能有哪些？
9. 活动文档与动态文档的区别是什么？
10. CGI的含义是什么？有什么功能？
11. 描述Java与活动文档之间的关系。

练习题

12. 试说出下面每一个图将会显示在屏幕上的什么位置？

Look at the following picture:

Then tell me what you feel:

```
<IMG SRC="Pictures/Funny1.gif" ALIGN=middle>
```

```
<IMG SRC="Pictures/Funny2.gif" ALIGN=bottom>
```

```
<B> What is your feeling? </B>
```

13. 说明下列HTML段落的效果：

```
The publisher of this book is <A HREF="www.mhhe">
```

```
McGraw-Hill publisher</A>
```

14. 写出检索文档/usr/users/doc/doc1的请求。至少使用2个通用头部、2个请求头部和一个实体头部。
15. 写出练习14成功请求的响应。
16. 写出练习14的响应，若文档已经永远地移动到/usr/deads/doc1。
17. 写出练习14的响应，如果在请求中有语法错误。
18. 写出练习14的响应。如果客户端未被授权访问这个文档。
19. 写出一个请求，用来查询在/bin/users/file的文档信息，请至少使用2个通用头部和一个请求头部。

20. 写出练习19成功请求的响应。
21. 写出将位于/bin/usr/file1的文件复制到/bin/file1的请求。
22. 写出练习21的响应。
23. 写出位于/bin/file1的文件删除的请求。
24. 写出练习23的响应。
25. 写出检索位于/bin/etc/file1的文件的请求。仅当这个文档在1999年1月23日之后修改过，客户端才需要这份文档。
26. 写出练习25的响应。
27. 写出检索位于/bin/etc/file1的文件的请求。客户端需要标识自己。
28. 写出练习27的响应。
29. 写出在/bin/letter存储文件的请求。客户端标识它可以接受的文档的类型。
30. 写出练习29的响应。响应必须说明了文档的使用时限，以及内容可能改变的日期和时间。

第28章 网络管理

我们将网络管理（network management）定义为监视、检测、配置和故障发现及修理的网络组件，以满足组织机构所规定的一组要求。这些要求能使网络顺利的、高效的运行，为用户提供预定的服务质量。为了达到这个任务，网络管理系统使用软件、硬件以及人工参与来实现。本章首先简要讨论网络管理系统的功能，然后介绍最常用的网络管理系统，简单网络管理协议（SNMP）。

28.1 网络管理系统

我们说网络管理系统的功能可划分5大类：配置管理、故障管理、性能管理、安全管理和计费管理。如图28.1所示。

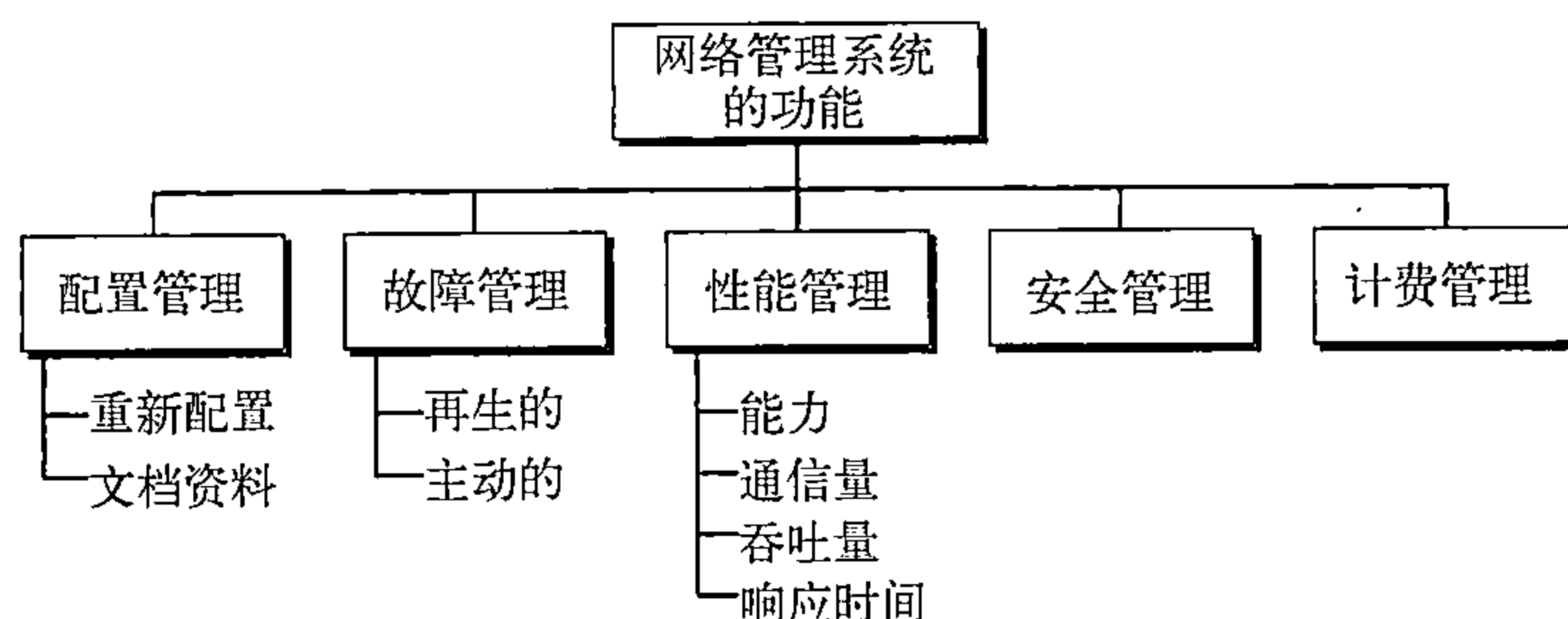


图28.1 网络管理系统的功能

28.1.1 配置管理

大型网络通常是由上千个物理地或逻辑地相互连接在一起的实体组成。在网络建立时，这些实体有一个初始的配置，但随着时间可能会改变。台式计算机可能由其他计算机取代，应用软件可能更新为新的版本，用户也可能由一组移动到另一组。任何时候，配置管理（configuration management）系统必须知道每个实体的状态及其与其他实体的关系。配置管理可划分成两个子系统：重新配置和文档资料。

重新配置

重新配置，是指调整网络组件与特性，在大型网络可能每天都会发生。重新配置有3种类型：硬件重新配置、软件重新配置和用户账号重新配置。

硬件重新配置包括硬件的所有变化。例如，台式计算机可能被其他计算机取代，一个路由器可能需要从网络的一个部分移动到另一个部分，某一个子网可能增加到网络上或从网络上删除。所有这些都需要网络管理及时关注。在大型网络中，必须有专职技术人员快速和高效地进行重新配置。令人遗憾的是，这种类型的重新配置不是自动的，而需按情况手工处理。

软件重新配置包括软件的所有变化。例如，新软件可能需要安装在服务器或客户机上，操作系统需要更新。幸好，大多数的软件重新配置都是自动的。例如，更新某些或所有客户机上的一个应用程序可以从服务器上下载电子版本。

用户账号重新配置不是简单地在系统上增加或删除用户，还必须考虑用户作为个人和作为一组的成员用户权限。例如，某个用户可以对有些文件有读和写的权限，但对另些文件仅有读

的权限。某种程度上，用户账号重新配置是自动的。例如，在学院或大学中，每一季度或每一学期开始将新生增加到系统中。根据新生所选择的课程或所从事的专业，通常将他们分成组。

文档资料

原始网络配置和每次变化都必须有详细的记录。这是说要有硬件、软件 and 用户账号文档。

硬件文档 (Hardware documentation) 通常包含两组文档：映照和说明。映照追踪硬件的每一部件及其与网络连接，也可有一个总的映照说明每个子网之间的逻辑关系。还可有第二个总映照，它说明了每个子网的物理位置。然后，对于每个子网有一个或多个说明了所有设备的映照。这些映照使用的标准能被当前的或以后的人们易于阅读和理解。仅有映照还是不够的，硬件的每一部件也需要有文档。连接网络的每一部件必需有一组说明，这些说明包含部件类型、序列号、制造厂商（地址和电话号码）、购买日期和保修卡等信息。

所有软件也需要有记录，软件文档包含的信息有软件型号、版本、安装日期和许可证。

多数操作系统有一个允许进入用户账号和权限的文件的程序，管理人员必须确保对这些文件进行更新与维护。有些操作系统在二个文档中记录访问权限：一个表示所有文件和每一个用户的访问类型；另一个表示访问一个具体文件的用户列表。

28.1.2 故障管理

当前复杂的网络是由数百个有时是数千个组件组成的，网络的正确运行取决于各个组件的正确运行以及相互之间的正确运行。**故障管理** (Fault management) 是处理这个问题的网络管理。

一个有效的故障管理系统有两个子系统：再生的故障管理和主动故障管理。

再生的故障管理

再生的故障管理系统负责故障的检测、隔离、纠正和记录。它处理对故障的短期解决办法。

故障定义为系统的一种不正常的工作情况。再生的故障管理系统所采取的第一步是检测故障准确的位置。当一种故障发生的时候，系统或完全停止工作或产生过多的错误。故障的好例子是通信介质被损坏。故障可能中断通信或产生过多的差错。

再生的故障管理系统所采取的第二步是隔离故障。如果故障被隔离，则通常仅有少数用户受影响。隔离后，立即通知受影响的用户并给出一个纠正错误的大致时间。

第三步是纠正故障，这可包含替换和修理故障的部分。

故障纠正后，必须被记录在文档中。这个记录必须表明故障的准确位置，可能引起的原因或修复故障的措施、成本和每步化费的时间。文档极其重要有下列几点理由：

- 故障可能重复发生，文档有助于网络管理人员或技术人员解决同样的类似问题；
- 同一类故障的频率是系统中较重要问题的指示。如果某一故障在一个组件中经常发生，就应该用另一个组件去更换它，或者改变整个系统不要使用那种组件；
- 统计是对网络管理和性能管理有帮助的另一个部分。

主动的故障管理

主动的故障管理是防止故障发生。尽管通常这是不可能的，但有些故障还是可预知的和可制止的。例如，制造商指定一个部件或一个部件中某部分的有效期，在到达有效期之前，更换它是一种好的策略。再例如，如果在网络某一特定点经常有故障发生，那么在重新配置网络时注意防止故障再一次发生也是明智的。

28.1.3 性能管理

与故障管理有着密切关系的是性能管理 (Performance management)，它试图监视和控制网络，确保网络尽可能有效地运行。性能管理试图通过一些可测量的量（如能力、通信量、吞吐量和响应时间等）来量化性能。

能力

性能管理系统必须监视的一个要素是网络的能力。每个网络的能力都是有限的，性能管理系统必须确保使用不能超过这个能力。例如，如果一个局域网设计为100个站点的平均数据速率是2Mbps，则连接到这个网络200个站点就不能正确地运行，数据速率将会下降甚至还可能发生拥塞。

通信量

通信量按两种方法测量：内部的和外部的。内部通信量是用网络内部传送分组（或字节）的个数来度量；外部通信量是用网络与外部交换分组（或字节）的个数来度量。系统在负载重的峰值期间，如果出现过多的通信量，拥塞就会发生。

吞吐量

我们可测量单独一台设备（如路由器）或部分网络的吞吐量。性能管理监视吞吐量确保它不降低于不可接受的水平。

响应时间

通常将从用户请求服务到服务被准许的时间称为响应时间。其他一些因素如能力和通信量会影响响应时间。性能管理监视平均响应时间和峰值时刻响应时间。

响应时间的增加是一个非常严重的情况，它表示网络正朝着超过它的能力方向工作。

28.1.4 安全管理

安全管理 (Security management) 是根据预定的负责策略监视对网络的访问。在第31章和第32章将讨论安全问题，尤其是网络安全问题。

28.1.5 计费管理

计费管理 (Accounting management) 是通过费用控制用户访问网络的资源。根据计费管理，个人用户、部门、公司，甚至工程等都需要对从网络上得到服务的付费。付费不必现金支付，为预算目的它可以记入部门或公司的账单上。今天，组织机构使用计费系统有下列原因：

- 它防止用户独占有限的网络资源；
- 它防止用户无效地使用系统；
- 网络管理员可以按需对网络使用作出短期和长期的计划。

28.2 简单网络管理协议 (SNMP)

简单网络管理协议 (SNMP) 是使用TCP/IP协议族对互联网上的设备进行管理的一个框架，它提供一组基本的操作来监控和维护互联网。

28.2.1 概念

SNMP使用管理器和代理的概念。这就是说，管理器，通常是一台主机，控制和监视一组代理，代理通常是路由器（如图28.2所示）。

SNMP是应用级协议，它用少数几个管理器控制一组代理。该协议设计在应用级，因此它能监控不同厂商制造的设备，而这些设备可安装在不同的物理网络上。换言之，SNMP使管理任务与被管理设备的物理特性和联网技术没有关系。它可以用于由不同厂商制造的路由器相互连接在一起的不同局域网和广域网组成的异构互联网中。

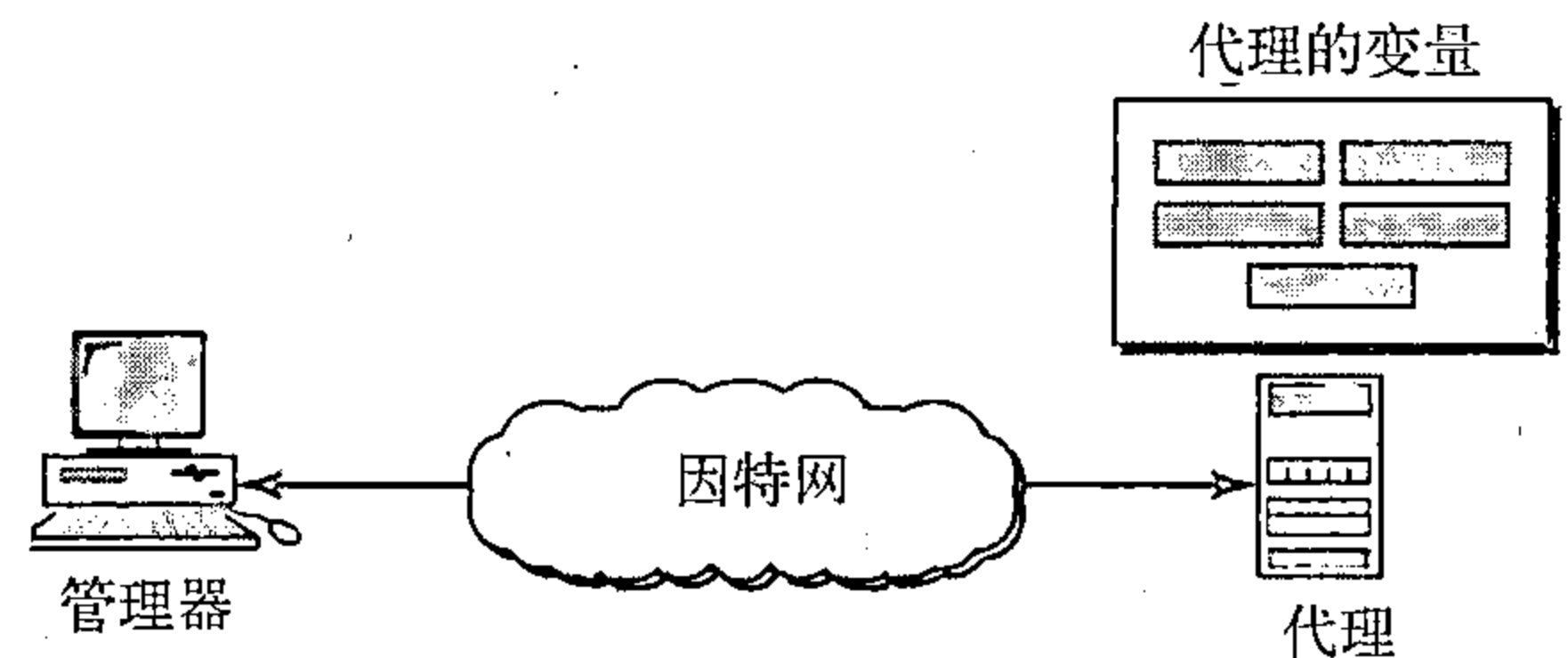


图28.2 SNMP概念

管理器和代理

称为管理器（manager）的管理站是运行SNMP客户程序的主机。称为代理（agent）的被管理站是运行SNMP服务器程序的路由器（或主机）。管理是通过管理站与代理之间的简单交互来实现的。

代理在数据库中保存性能信息，管理器使用该数据库中的值。例如，路由器可将接收到的和转发的分组数存储成适当的变量。管理器可读取和比较这两个变量，以便确定路由器是否拥塞。

管理器也可使路由器完成某些动作。例如，路由器定期检查一个重新启动计数器的值，看它何时应当重新启动自己。例如，当计数器的值为0时就应该引导自己。管理器可使用这个特性在任何时候从远程重新引导这个代理，它只要发送一个分组，就迫使这个计数器的值为0。

代理也可参加到管理过程中。在代理上运行的服务器程序可检查环境，如果发现异常现象可发送一个称为陷阱（trap）的告警报文给管理器。

换言之，SNMP管理是基于3个基本思想：

1. 管理器检查代理的方法是发出请求得到能反映代理行为的信息；
2. 管理器可用重新设置在代理数据库中的某些值来强迫代理完成一个任务；
3. 代理管理过程的方法是向管理器发送异常情况的告警。

28.2.2 管理组件

为了完成管理任务，SNMP还使用另外两个协议：管理信息结构（Structure of Management, SMI）和管理信息库（Management Information Base, MIB）。换言之，在因特网上的管理是通过SNMP、SMI和MIB三个协议共同协作完成的，如图28.3所示。

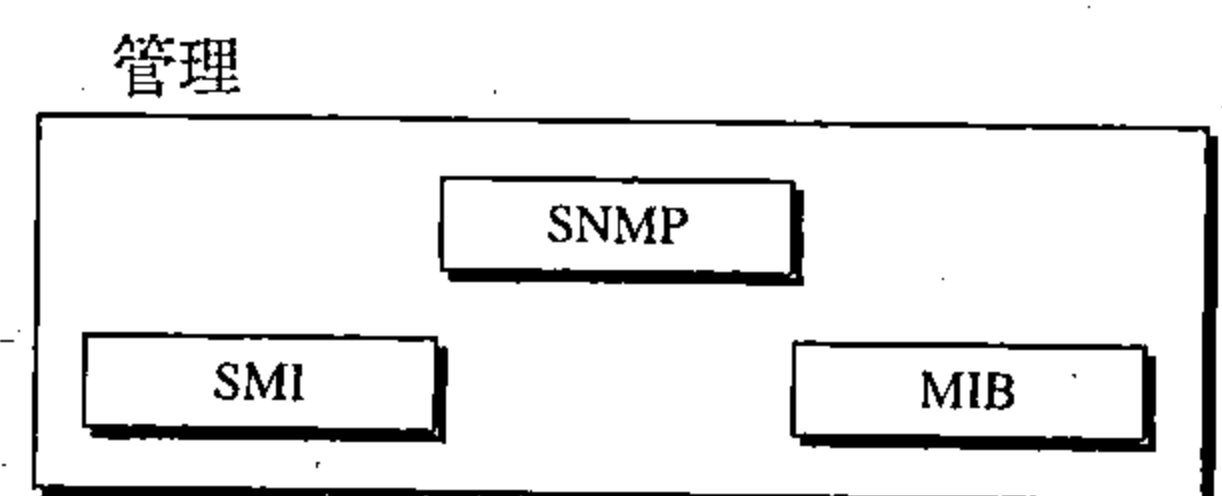


图28.3 因特网上网络管理组件

让我们详细阐述三个协议之间的交互作用。

SNMP的作用

SNMP在网络管理中起着一些很特殊的作用，它定义从管理器到代理和从代理到管理器发送分组的格式。还解释产生的结果并创建统计表（时常借助其他的管理软件）。交换的分组包括对象（变量）名及它们的状态（值）。SNMP负责读取和改变这些值。

SNMP定义管理器与代理之间交换分组的格式，读取和改变SNMP分组中对象（变量）的状态（值）。

SMI的作用

要使用SNMP，我们需要有一些规则，需要命名对象的规则。因为在SNMP中的对象是有

层次结构的（对象有一个父对象和多个子对象），所以命名规则特别重要。对象名字的一部分可继承父对象的名字。还须有规则定义对象的类型。SNMP能处理什么样的类型？SNMP能否处理简单类型还是结构类型？这些类型的定义域是什么？这些类型如何编码？

因为我们不知道发送、接收或存储这些值的计算机的结构，我们必须有通用的规则。例如，发送计算机是一个大型计算机，能将整数存储为8字节数据，而接收计算机是一个小型的计算机，仅能将整数存储为4字节数据。

SMI是定义这些规则的协议。但我们必须明白SMI仅是定义规则，不能定义一个实体管理多少个对象或对象使用什么类型。SMI是给对象命名和定义对象数据类型的规则集合。对象与数据类型的关联不由SMI来做出。

SMI定义对象命名、数据类型（包括长度和定义域）以及编码方法和取值的一般规则。
SMI不定义实体应管理多少个对象或被管理对象的命名，或对象与值之间的关联。

MIB的作用

我们期望清晰地了解另一个所需协议。对每个被管理的实体，这个协议必须按照SMI规定的规则定义对象的个数和给对象命名，以及每个对象与类型的关联。这就是MIB协议，与数据库（在数据库中，大多数元数据未命名和没有类型）类似，MIB对每个实体创建一组规定的对象。

MIB创建一个被管理的实体中所有对象命名和类型，以及它们之间相互关系。

类比

在较详细讨论这三个协议的之前，我们先给出一个类比。这三个网络管理组件类似于为求解某个问题用计算机语言所编写的程序。

在编写程序前，必须预先定义程序设计语言的语法（如C语言或Java语言），语言还定义变量的结构（简单的、结构的、指针等）以及如何给变量命名。例如，一个变量名字长度需要1到N个字符，以一个字母开始后跟数字、字母字符。语言还定义所用的数据类型（整数型、浮点型、字符型等）。在程序设计中，规则是由语言来定义的。而在网络管理中，规则是由SMI来定义。

大多数程序设计语言要求在每个具体的程序中声明变量，其内容包括变量的名字及其类型。例如，如果某程序有二个变量：一个名字为`counter`，数据类型是整数型；另一个名字为`grades`，数据类型是字符型，那么在程序开始时这二个变量必须声明：

```
int counter;  
char grades [40];
```

注意：声明给变量命名（`counter`和`grades`）和定义每个变量的数据类型。由于在程序设计语言中，数据类型是预先定义的，所以程序知道每个变量的定义域与大小。

MIB在网络管理中负责类似的任务，MIB给每个对象命名并定义其类型。因为SMI定义类型，所以SNMP知道其定义域与大小。

在编制程序的声明后，程序要编写语句存储变量的值，如有需要，还要更新其值。SNMP在网络管理中负责类似的任务，SNMP按SMI定义规则对MIB已说明的对象值进行存储、更新和解释。

我们比较网络管理任务与编写程序的任务。

• 两个任务都需要规则。在网络管理中，它由SMI处理。